

A differentiable high-order discontinuous Galerkin solver for incompressible Navier–Stokes: automating numerical-parameter tuning, shape optimization, and data assimilation

Corbin Foucart

Abstract

We describe an implementation of a high-order discontinuous Galerkin (DG) method for the incompressible Navier–Stokes equations in which every operator of the forward solver, including the implicit Newton inner solve and the curved-element mesh constructor, is traced by reverse-mode automatic differentiation. The underlying discretization uses the $H(\text{div})$ -conforming BDM_k/P_{k-1} velocity–pressure pair and BDF1/BDF2 time integration following the convention of [10]. To our knowledge this is the first end-to-end differentiable framework for the incompressible Navier–Stokes equations built on a high-order, $H(\text{div})$ -conforming, pressure-robust DG discretization, with gradients obtained by reverse-mode automatic differentiation through the implicit solves rather than a hand-coded or automated discrete adjoint. We use this to address four problem classes that previously required hand-derived adjoint code or expensive finite-difference sensitivity analysis. First, several numerical parameters that are routinely set by heuristic (the time step, the SIPG penalty coefficient, the relaxation weights in a multigrid V-cycle) become explicit differentiable design variables and can be jointly optimized by stochastic gradient descent through the implicit time-stepping loop. Second, boundary control of a cylinder wake reduces the downstream wake enstrophy: a mass-conserving learned inflow schedule lowers it by 12%, while a body-mounted zero-net-mass-flux jet, an actuator with far greater control authority, reaches 72.9%. Third, drag-minimizing shape optimization is posed directly on the geometric parameters of a curved-element mesh, sidestepping both the discrete-alignment artifacts of finite-difference-on-rebuild and the level-set / hand-coded-adjoint formalisms standard in this setting. Fourth, 4D-VAR initial-condition recovery of a 3,354-dimensional velocity vector from sparse downstream probe observations is driven by Adam, with the gradient supplied by the implicit-time-stepping adjoint pass at cost independent of parameter count. The solver is validated against the Stokes pressure-robustness sentinel, Kovasznay convergence, the Ghia lid-driven cavity at $\text{Re} \in \{100, 400\}$, and the Schäfer–Turek cylinder benchmark. A block-Schur projection variant of the solver runs up to $\sim 21\times$ faster per step than the monolithic Newton ($\sim 4\text{--}5.5\times$ at the finest meshes of the scaling sweep) and reproduces its accuracy (the same design-order spatial and temporal convergence, with trajectory fields agreeing to a relative L^2 of 5×10^{-5}) within an empirically characterized explicit-convection stability envelope.

1 Introduction

High-order discontinuous Galerkin (DG) methods for the incompressible Navier–Stokes equations have developed substantially over the last three decades, from the nodal-DG presentation [16] through early pressure-correction schemes [19] to a body of more recent work on $H(\text{div})$ -conforming pressure-correction [10, 12, 13]. Three contributions are most relevant to the present work: a consistent-splitting treatment [12] that resolves the small- Δt instability of the standard dual splitting; divergence- and continuity-penalty terms [13] that restore mass conservation under coarse-mesh ILES-style use; and an $H(\text{div})$ -conforming BDM_k/P_{k-1} formulation [10] that is discretely mass-conservative. Together with the ExaDG [11] and deal.II [2] implementations these references constitute a mature solver technology for transitional and turbulent incompressible flow at the moderate-to-high Reynolds numbers typical of engineering applications.

In parallel, machine-learning toolchains have produced flexible optimization frameworks for problems with many parameters. The combination of reverse-mode automatic differentiation (AD), gradient-based optimizers such as Adam and L-BFGS, and just-in-time compilation has made it routine to optimize over high-dimensional design spaces by treating a numerical model as a smooth function and querying its gradient.

Within fluid dynamics, this has been applied predominantly to low-order finite-difference and finite-volume forward solvers [3, 21] and to surrogate-replacement approaches such as physics-informed neural networks [24]. Shape-optimization historically uses hand-coded adjoints of the discrete solver [1]; classical 4D-VAR for data assimilation [22, 27] similarly relies on hand-derived tangent-linear and adjoint models. Learned relaxation parameters for multigrid solvers have been pursued through meta-learning with an external surrogate [14] and through direct backpropagation through the V-cycle.

Combining these two threads (a high-order $H(\text{div})$ -conforming DG-INS solver and the AD-based optimization framework typical of modern machine learning) has not, to our knowledge, been attempted as a single integrated implementation. The structural obstacles include: the Piola map carries a per-element Jacobian through every basis evaluation; the cross-space gradient and divergence operators have asymmetric trace conventions arising from $H(\text{div})$ -conformity; the Lax–Friedrichs convective flux contains a non-smooth $|u \cdot n|$ requiring a subgradient choice; and the implicit time integration that the Reynolds-number regime requires adds an inner Newton solve whose Jacobian must be inverted in the backward pass. The mesh constructor itself is, in most existing high-order DG implementations, written as imperative code that operates on numerical arrays rather than as a differentiable function of its geometric inputs.

The premise of this paper is that the relatively modest, one-time engineering effort of integrating automatic differentiation into a mature high-order solver opens a broad range of design, control, and inference applications at no cost to the underlying numerics. We describe an implementation of a BDM_k/P_{k-1} DG-INS scheme inspired by the stability and robustness advances of [10] in a framework where every operator is traced by reverse-mode AD, including the implicit Newton inner solve through the implicit-function theorem and the curved-element mesh constructor through differentiable geometry. The benefits, demonstrated by example, fall into two categories. *First*, several numerical parameters that are routinely tuned by hand or by heuristic (the time step at the edge of stability, the SIPG penalty τ , the Jacobi relaxation weights in a multigrid V-cycle, the Brezzi–Pitkäranta pressure-stabilization constant) become explicit differentiable design variables. Stochastic gradient descent over a short fixed-shape time-stepping trajectory locates the largest stable Δt , identifies per-element penalty constants that depart from the uniform- τ heuristic on curved-boundary regions, and learns multigrid smoother weights that reduce CG iteration counts by $\sim 1.6\times$ relative to the unit-weight default. *Second*, the same differentiable forward solver becomes a tool for control and experimentation: drag-minimizing shape optimization through a Fourier-mode parameterization of a cylinder’s boundary converges in 10 Adam iterations with no hand-coded shape derivatives; 4D-VAR recovery of a 3,354-dimensional initial velocity vector from 48 scalar observations drives the observation loss down at cost independent of parameter count. The unifying observation is that turning a solver into a differentiable function is what makes these applications tractable, and that doing so on a mature high-order discretization requires no compromise of the underlying numerical method.

The present work synthesizes elements from several adjacent lines of research. The discretization follows [10] closely; the block-Schur factorization is the classical Cahouet–Chabard approach [8] applied to the BDM_k/P_{k-1} pair. The differentiable applications draw on the differentiable-CFD lineage [3, 21], the shape-derivative shape-optimization literature [1], classical 4D-VAR [22, 27], and learned multigrid relaxation [14]. The differentiable flow solvers nearest to ours occupy adjacent cells of the design space rather than the same one: end-to-end differentiable *high-order DG* solvers have so far targeted the *compressible* equations [18]; the established differentiable *incompressible* solvers are *low-order* finite-volume or finite-difference [3, 21]; and automated-adjoint finite-element frameworks [25] obtain gradients by assembling and solving the discrete adjoint *equations*, typically over low-order continuous elements, rather than by reverse-mode AD through the solver itself. None of the ingredients we use is individually new. The contribution is, to our knowledge, the first end-to-end differentiable framework for the *incompressible* Navier–Stokes equations built on a high-order, $H(\text{div})$ -conforming, pressure-robust DG discretization: the geometry, the implicit time-stepping, and the multigrid V-cycle are all traced by reverse-mode AD, so a single backward pass differentiates the unmodified solver rather than a hand-coded or automated discrete adjoint.

The contributions of the paper, by section, are as follows. We give a self-contained presentation of the BDM_k/P_{k-1} DG-INS discretization following [10] (Sections 2 to 4), including the monolithic Newton solver and a block-Schur projection variant that runs up to $\sim 21\times$ faster per step (§5). We describe the differentiable formulation of the full forward simulation, including the implicit Newton step and the curved-element mesh constructor (§6), and the implementation choices that make this efficient (§7). We verify the solver against the standard benchmarks (§8) and report the differentiable applications (automated tuning of three classes of

numerical parameter, shape optimization, and 4D-VAR) in §9. Section 10 closes with limitations and open extensions.

2 Continuous problem

We consider the unsteady incompressible Navier–Stokes equations on a bounded domain $\Omega \subset \mathbb{R}^2$ over a time interval $[0, T]$, with boundary $\partial\Omega = \Gamma_D \cup \Gamma_N$:

$$\begin{aligned} \partial_t u + \nabla \cdot (u \otimes u) - \nu \Delta u + \nabla p &= f & \text{in } \Omega \times [0, T], \\ \nabla \cdot u &= 0 & \text{in } \Omega \times [0, T], \\ u &= u_D & \text{on } \Gamma_D \times [0, T], \\ (\nu \nabla u - p \mathbf{I}) \cdot n &= 0 & \text{on } \Gamma_N \times [0, T], \\ u &= u_0 & \text{in } \Omega, \ t = 0. \end{aligned} \tag{1}$$

Here $u: \Omega \times [0, T] \rightarrow \mathbb{R}^2$ is the velocity and $p: \Omega \times [0, T] \rightarrow \mathbb{R}$ the pressure (the unknowns); $\nu > 0$ is the kinematic viscosity; f a body force; $\mathbf{I}$ the 2×2 identity tensor; n the outward unit normal on $\partial\Omega$. The differential operators are ∂_t (time derivative), ∇ (spatial gradient), $\nabla \cdot$ (divergence), and $\Delta = \nabla \cdot \nabla$ (the componentwise vector Laplacian); ∇u is the velocity-gradient tensor with $(\nabla u)_{ij} = \partial_j u_i$, $(\nabla u) n$ its contraction $((\nabla u) n)_i = \sum_j (\nabla u)_{ij} n_j$ with the normal, and $u \otimes u$ the rank-two tensor $(u \otimes u)_{ij} = u_i u_j$. The Reynolds number is $\text{Re} = UL/\nu$ with U and L a characteristic velocity and length specified per benchmark (e.g. for the cylinder U is the mean inflow speed and L the cylinder diameter). The remaining data are u_D the prescribed Dirichlet velocity on Γ_D (inflow, walls, obstacles); Γ_N the free-stress outflow boundary; and u_0 the initial velocity. Optional periodic identifications of $\partial\Omega$ are also supported. On an all-Dirichlet boundary ($\Gamma_N = \emptyset$) the pressure is determined up to an additive constant.

3 Discretization

3.1 Finite-element spaces

The velocity is discretized in the Brezzi–Douglas–Marini space $\text{BDM}_k(\mathcal{T}_h)$ [6] on a conforming triangulation $\mathcal{T}_h$ of Ω , and the pressure in the discontinuous polynomial space $P_{k-1}^{\text{DG}}(\mathcal{T}_h)$. Following [10],

$$\begin{aligned} V_h &= \{ u \in H(\text{div}; \Omega) : u|_K \in P_k(K)^2 \ \forall K \in \mathcal{T}_h \}, \\ Q_h &= \{ q \in L^2(\Omega) : q|_K \in P_{k-1}(K) \ \forall K \in \mathcal{T}_h \}. \end{aligned} \tag{2}$$

Here $L^2(\Omega)$ is the space of square-integrable functions on Ω (and $L^2(\Omega)^2$ its vector-valued analogue), $H(\text{div}; \Omega) = \{ w \in L^2(\Omega)^2 : \nabla \cdot w \in L^2(\Omega) \}$ the space of square-integrable vector fields with square-integrable weak divergence, and $P_m(K)$ the space of polynomials of total degree at most m on element K ; the superscript $^{\text{DG}}$ marks a broken space with no inter-element continuity imposed, and $(\cdot)^2$ the componentwise vector-valued version. We write $|\mathcal{T}_h|$ for the number of elements. The BDM_k degrees of freedom on element K are the normal-trace values $u \cdot n$ at $k+1$ Gauss–Legendre points on each edge ($3(k+1)$ face DOFs total, shared with the neighbor) plus $(k-1)(k+1)$ interior moments against an internal test basis. The reference basis $\{\varphi_\ell^{\text{ref}}\}_\ell$ (with ℓ indexing the basis functions and the superscript $^{\text{ref}}$ marking a quantity on the reference element $\hat{K}$) is constructed by inverting the DOF Vandermonde against a monomial basis of $P_k(K)^2$, then Piola-mapped to physical space,

$$\varphi_{K,\ell}^{\text{phys}}(x) = \frac{1}{J_K} DF_K \varphi_\ell^{\text{ref}}(\hat{x}), \quad x = F_K(\hat{x}), \tag{3}$$

with $\hat{x} \in \hat{K}$ the reference coordinate, $F_K: \hat{K} \rightarrow K$ the (affine or curved) element map, DF_K its Jacobian matrix, and $J_K = |\det DF_K|$. The Piola map preserves the normal-trace value $u \cdot n$ and the strong divergence; the divergence of any BDM_k field lies pointwise in P_{k-1} and is exactly representable in Q_h . The one-degree-lower pressure is a discrete inf-sup (LBB) stability requirement, not a convenience: the divergence maps V_h onto Q_h exactly, so P_{k-1} is the richest pressure space the velocity divergence can control. We also tried the

equal-order BDM_k/P_k pair and found it inf-sup-unstable: the pressure modes beyond the velocity divergence go uncontrolled and the discrete saddle becomes singular.

Throughout we use two L^2 pairings, distinguished by bracket shape. Rounded brackets denote the d -dimensional *interior* inner product over a region ω , and angled brackets the $(d-1)$ -dimensional *face* inner product over a face or face set γ ,

$$(a, b)_\omega = \int_\omega a \cdot b \, dx, \quad \langle a, b \rangle_\gamma = \int_\gamma a \cdot b \, ds, \quad (4)$$

both reducing to the scalar product of the integrands and applied component-wise to vector- or tensor-valued arguments (so for tensors $a \cdot b$ is the Frobenius contraction $\sum_{ij} a_{ij} b_{ij}$). We write $(\cdot, \cdot)_\Omega = \sum_K (\cdot, \cdot)_K$ for the interior product over the whole domain, and $\sum_K, \sum_F$ abbreviate sums over all elements $K \in \mathcal{T}_h$ and over a face set specified at each use. The discrete divergence operator $D: V_h \rightarrow Q_h$ is therefore element-local and surjective onto $Q_h/\mathbb{R}$ (the pressure space modulo additive constants, the constant-pressure nullspace on closed-domain problems with homogeneous Dirichlet velocity). The discrete gradient $G: Q_h \rightarrow V_h$ is the negative adjoint $-D^\top$ of D in the interior pairing, so that $(Gp, u)_\Omega = -(p, Du)_\Omega$ for all $u \in V_h, p \in Q_h$. Both operators share the same reference-element matrix, assembled in closed form by integration of $\nabla \cdot \varphi_\ell^{\text{ref}}$ against the P_{k-1} basis functions.

3.2 Body-fitted curved mesh

The cylinder is resolved by a body-fitted O-grid: an inner annulus of n_r radial layers fills the region between the cylinder and a grid-aligned bounding square of half-side R_{out} , and the Cartesian channel mesh fills the exterior with that square excised, the two regions sharing vertices along the square's perimeter. The innermost ring is built with its inner vertices on the cylinder but its inner faces as straight chords; these faces are then bent onto the true arc, so the layer of elements abutting the cylinder is *curved*: isoparametric of the same degree k as the velocity. Figure 1 sketches the element map.

On a curved element the map $F_K: \hat{K} \rightarrow K$ is no longer affine, so its Jacobian $DF_K(\hat{x})$, the determinant $J_K(\hat{x}) = \det DF_K(\hat{x})$, and the inverse metric $DF_K^{-1}(\hat{x}) = \partial(\hat{r}, \hat{s})/\partial(x, y)$ all vary across the element. We therefore evaluate every element integral on a cubature rule exact for the integrand degree, carrying J_K and DF_K^{-1} at each cubature point. Physical derivatives of a scalar ϕ follow from the chain rule with the per-point metric,

$$\partial_x \phi = r_x \partial_{\hat{r}} \phi + s_x \partial_{\hat{s}} \phi, \quad \partial_y \phi = r_y \partial_{\hat{r}} \phi + s_y \partial_{\hat{s}} \phi, \quad (5)$$

with (r_x, s_x, r_y, s_y) the entries of DF_K^{-1} at that point, and the BDM velocity basis is Piola-mapped pointwise through (3). The Piola map carries the normal trace and the strong divergence through the curved map undistorted, so the discrete divergence of a BDM_k field stays pointwise in P_{k-1} even on the curved boundary layer.

Because F_K is assembled from the geometric parameters (here the cylinder center and radius) by differentiable operations, the metric quantities DF_K , J_K , and DF_K^{-1} are themselves differentiable functions of those parameters. The sensitivity $\partial(\cdot)/\partial(\text{geometry})$ propagates through the mesh into the discrete operators, the mechanism the shape-optimization demonstration of §9.4 exploits.

3.3 Spatial discretization

Trace and face notation. Let $\mathcal{F}_h = \mathcal{F}_h^i \cup \mathcal{F}_h^\partial$ be the set of mesh faces (edges in 2D), split into interior faces $\mathcal{F}_h^i$ and boundary faces $\mathcal{F}_h^\partial = \mathcal{F}_h^D \cup \mathcal{F}_h^N$ on the Dirichlet (Γ_D) and natural/outflow (Γ_N) parts of $\partial\Omega$. An interior face $F \in \mathcal{F}_h^i$ is shared by two elements; we label them K^- and K^+ , and let $n \equiv n^-$ be the unit normal of F pointing out of K^- (so $n^+ = -n^-$). The two one-sided traces of a (generally discontinuous) field w on F are then defined by the normal limits

$$w^\pm(x) = \lim_{s \rightarrow 0^+} w(x \mp sn), \quad x \in F, \quad (6)$$

so that w^- is the value of w at x approached from within K^- (the interior trace, since $x - sn \in K^-$ for small $s > 0$) and w^+ the value approached from within K^+ (the neighbor trace). The average and jump operators are

$$\{\{w\}\} = \frac{1}{2} (w^- + w^+), \quad \llbracket w \rrbracket = w^- - w^+, \quad (7)$$

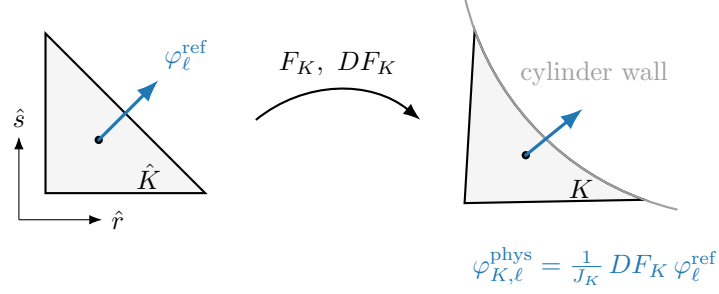


Figure 1: Curved element map. The reference triangle $\hat{K}$ (left) is mapped to a body-fitted element K (right) whose edge on the cylinder follows the true arc; for boundary elements F_K is non-affine, so the Jacobian DF_K , J_K , and metric DF_K^{-1} vary within K and are evaluated per cubature point (Equation (5)). The BDM velocity basis is carried by the Piola map (Equation (3)), preserving the normal trace and the strong divergence on curved elements.

applied component-wise to vectors and row-wise to the tensor ∇u . On a boundary face $F \in \mathcal{F}_h^\partial$ the neighbor side is supplied by a ghost state: $w^+ = 2w_{\text{BC}} - w^-$ on Γ_D (mirror ghost enforcing $\{\{w\}\} = w_{\text{BC}}$), and $w^+ = w^-$ on Γ_N (so interior penalties vanish), where w_{BC} is the prescribed boundary value of the field w on that face (for the velocity, $w_{\text{BC}} = u_D$ of (1) on Γ_D). Finally, h_F denotes the diameter (edge length) of face F , and h_K the diameter of element K .

The semi-discrete momentum equation, integrated against $v \in V_h$, has the form

$$(\partial_t u, v)_\Omega + a^{\text{conv}}(u_*; u, v) + \nu a^{\text{SIPG}}(u, v) + (Gp, v)_\Omega = (f, v)_\Omega + \text{lift}_{\text{BC}}(v), \quad (8)$$

and the discrete divergence constraint is

$$(Du, q)_\Omega = 0 \quad \forall q \in Q_h. \quad (9)$$

Here u_* is the (known) transport velocity at which the convective term is linearized, taken explicitly from previous time levels by the BDF extrapolation (15) of §4; the semicolon in $a^{\text{conv}}(u_*; u, v)$ separates this frozen advecting field from the trial/test arguments (u, v) , in which the form is bilinear. The boundary-only lift $\text{lift}_{\text{BC}}(v)$ carries the prescribed Dirichlet data and is given explicitly in (11) below.

Viscous SIPG bilinear form. The symmetric interior-penalty treatment of $-\nu \Delta u$ is

$$a^{\text{SIPG}}(u, v) = (\nabla u, \nabla v)_\Omega - \sum_F \left[\langle \{\{ \nabla u \cdot n \} \}, [v] \rangle_F + \langle \{\{ \nabla v \cdot n \} \}, [u] \rangle_F - \tau \langle [u], [v] \rangle_F \right], \quad (10)$$

with $(\nabla u, \nabla v)_\Omega = \int_\Omega \nabla u : \nabla v \, dx$ the interior pairing (4) of the velocity-gradient tensors (Frobenius contraction $A : B = \sum_{ij} A_{ij} B_{ij}$), $\langle \cdot, \cdot \rangle_F$ the face pairing (4), $\{\{ \cdot \} \}$, $[\cdot]$ the average and jump operators of (7), and the face sum running over interior and Dirichlet faces $(\mathcal{F}_h^i \cup \mathcal{F}_h^D)$; on a Dirichlet face the mirror ghost of §3.3 gives $[u] = 2(u^- - u_{\text{BC}})$. The penalty parameter is $\tau = \tau_{\text{scale}}(k+1)^2/h_F$ with τ_{scale} an $O(1)$ constant (default 4) and h_F the face length. On free-stress faces all three face contributions vanish (the natural BC is absorbed by integration by parts). Because BDM_k is $\text{H}(\text{div})$ -conforming, the normal-trace jump $[u \cdot n]$ vanishes identically on interior faces and no normal-jump penalty is needed; (10) therefore acts only on the tangential trace.

Non-homogeneous Dirichlet data enters through the boundary-only lift $\text{lift}_{\text{BC}}(v)$ of (8). It collects exactly the terms of the Dirichlet-face sum in (10) that depend on the prescribed data u_{BC} : substituting the mirror ghost (so $[u] = 2(u^- - u_{\text{BC}})$) into the symmetry and penalty terms on $\mathcal{F}_h^D$ and moving the u_{BC} -dependent part to the right-hand side gives

$$\text{lift}_{\text{BC}}(v) = \nu \sum_{F \in \mathcal{F}_h^D} \left[2\tau \langle u_{\text{BC}}, [v] \rangle_F - 2 \langle \{\{ \nabla v \cdot n \} \}, u_{\text{BC}} \rangle_F \right], \quad (11)$$

with $\langle \cdot, \cdot \rangle_F$ the face pairing (4) and $[v]$, $\{\{ \nabla v \cdot n \} \}$ the same boundary-face quantities as in (10); the u^- -dependent remainder stays in $a^{\text{SIPG}}(u, v)$ on the left-hand side.

Convective Lax–Friedrichs flux. The convective bilinear form, in conservation form $\mathcal{N}(u) = \nabla \cdot (u \otimes u)$, is

$$a^{\text{conv}}(u_*, u, v) = -(u_* \cdot u, \nabla \cdot v)_\Omega + \sum_F \langle \mathcal{F}^{\text{LF}}(u_*, u), \llbracket v \rrbracket \rangle_F, \quad (12)$$

with the local Lax–Friedrichs numerical flux written in terms of the average/jump operators of (7),

$$\mathcal{F}^{\text{LF}}(u_*, u) = (u_* \cdot n) \{ \{u\} \} + \frac{1}{2} \lambda \llbracket u \rrbracket, \quad \lambda = \max(|u_* \cdot n^-|, |u_* \cdot n^+|), \quad (13)$$

the dissipation speed λ being the larger one-sided normal advection speed (equal to $|u_* \cdot n|$ when u_* is H(div)-conforming, as here). On Dirichlet boundary faces the neighbor trace u^+ is the prescribed BC value; on free-stress faces $u^+ = u^-$ (natural outflow). The volume integrand is degree $3k$ on the face, exceeding the exactness of the standard $k + 1$ -point Gauss–Legendre rule; we use $\lceil (3k + 1)/2 \rceil$ over-integration points, following the recipe of [10] which verifies convergence rate restoration to h^{k+1} for the BDM-DG Kovasznay test.

Penalty post-projection. The divergence- and continuity-penalty terms [13] can be applied as an optional post-projection correction. For BDM_k/P_{k-1} the divergence-penalty contribution is identically zero (the discrete velocity is already pointwise divergence-free) and only the continuity-penalty contributes; we retain it as an option in the framework and disable it for the H(div)-conforming results below.

4 Time integration and nonlinear treatment

We discretize in time with backward differentiation formulae of order $q \in \{1, 2\}$. Given history $u^n, u^{n-1}, \dots$ at time levels $t^n, t^{n-1}, \dots$,

$$\partial_t u|_{t^{n+1}} \approx \frac{1}{\Delta t} \left(\gamma_0 u^{n+1} - \sum_{i=0}^{q-1} \beta_i u^{n-i} \right), \quad (14)$$

with the BDF coefficients $(\gamma_0, \{\beta_i\}) = (1, \{1\})$ for $q = 1$ and $(3/2, \{2, -\frac{1}{2}\})$ for $q = 2$; here γ_0 multiplies the new level u^{n+1} and β_i weight the history u^{n-i} . BDF2 requires two history states; the first time level is taken with BDF1. Separately, the explicit treatment of convection uses the order- q *extrapolation* of the velocity to t^{n+1} ,

$$u_* = \sum_{i=0}^{q-1} \alpha_i u^{n-i}, \quad (\alpha_i) = (1) \text{ for } q = 1, \quad (\alpha_0, \alpha_1) = (2, -1) \text{ for } q = 2, \quad (15)$$

whose weights α_i differ from the BDF history weights β_i .

The convective term $\mathcal{N}(u)$ in (1) is the sole nonlinearity. We expose three treatments. *Explicit extrapolation* freezes the transport velocity in (12) at the extrapolated value u_* of (15), giving a per-step linear solve with no nonlinear inner loop; this is the standard IMEX treatment in dual and consistent splitting schemes. *Picard iteration* updates u_* each iteration to the current outer iterate u_k and repeats $u_{k+1} = u_k - A(u_k)^{-1} F(u_k, p_k)$, with $A(u_k)$ the saddle operator (16) evaluated at $u_* = u_k$ and F the saddle residual defined in §5.1, until $\|u_{k+1} - u_k\|_{L^2(\Omega)} < \text{tol}$ for a fixed velocity tolerance tol , converging linearly to the implicit-convection root. *Newton–Krylov* linearizes the convective volume and flux terms with respect to the transported velocity as well: writing $\partial_j = \partial/\partial x_j$, with summation over the repeated spatial index j and δu the velocity increment, the Picard block $\partial_j(u_j \delta u_i)$ is augmented by the transport-velocity term $\partial_j(\delta u_j u_i)$, so that $\mathcal{N}'(u) \delta u = \partial_j(u_j \delta u_i) + \partial_j(\delta u_j u_i)$. The Lax–Friedrichs dissipation speed $\lambda = \max(|u_* \cdot n^-|, |u_* \cdot n^+|)$ is *frozen* at the current iterate u_k and not differentiated; this is a deliberate Picard-type treatment of the (non-smooth) dissipation that keeps the linear operator well-defined at $u_* \cdot n = 0$ and is what the code implements. The remaining non-smoothness, which matters for the *reverse-mode gradient* of the converged solver rather than for the Newton iteration, is handled by the subgradient convention of §6.

Combining (8)–(9) with BDF q gives, at each step, the linear saddle-point system

$$\underbrace{\begin{bmatrix} \frac{\gamma_0}{\Delta t} M + \nu a^{\text{SIPG}} + \mathcal{C}(u_*) & G \\ -D & -\varepsilon I \end{bmatrix}}_{A(u_*)} \begin{bmatrix} u^{n+1} \\ p^{n+1} \end{bmatrix} = \begin{bmatrix} \text{lift}_{\text{BC}} + M f_{\text{hist}} \\ 0 \end{bmatrix}, \quad (16)$$

where the matrix of any bilinear form b is understood in the convention $B_{ij} = b(\varphi_j, \varphi_i)$ for the basis $\{\varphi_i\}_{i=1}^{n_u}$ of V_h , so that $v^\top B u = b(u, v)$ when u, v are identified with their coefficient vectors. In particular M is the BDM mass matrix $M_{ij} = (\varphi_i, \varphi_j)_\Omega$, the SIPG block has entries $a^{\text{SIPG}}(\varphi_j, \varphi_i)$, and the discrete convective block $\mathcal{C}(u_*)$ is the matrix of the convective bilinear form (12) at frozen transport velocity u_* ,

$$v^\top \mathcal{C}(u_*) u = a^{\text{conv}}(u_*; u, v) \quad \forall u, v \in V_h, \quad (17)$$

linear in the solution slot u . The remaining quantities are $f_{\text{hist}} = \sum_{i=0}^{q-1} (\beta_i / \Delta t) u^{n-i}$ the BDF history of (14), I the $n_p \times n_p$ identity on Q_h (distinct from the tensor identity $\mathbf{I}$), and $\varepsilon \sim 10^{-10}$ a Brezzi–Pitkäranta stabilization of the constant-pressure nullspace on closed domains; the block sizes are $n_u = \dim V_h$ and $n_p = \dim Q_h$. In the full-Newton treatment of §4 the $(1, 1)$ block carries in addition the transport-velocity Jacobian $\mathcal{C}'(u_*)$, the matrix of $(u, v) \mapsto \int \partial_j (u_j u_{*,i}) v_i$; the Picard treatment retains only $\mathcal{C}(u_*)$.

5 Solution methods

5.1 Monolithic Newton

The straightforward path through (16) is direct dense LU factorization of the assembled $(n_u + n_p) \times (n_u + n_p)$ saddle matrix, followed by back-substitution. Strong-Dirichlet enforcement is via interior reduction: BC rows of A are replaced with the identity, BC columns are zeroed in the interior rows, and the corresponding RHS entries are set to zero. The Newton outer loop uses the unified δ -form $A(u_k) \delta = -F(u_k)$, $u_{k+1} = u_k + \delta$, where $F(u) = A(u)[u; p] - b$ is the saddle residual (b the right-hand side of (16), $[u; p]$ the stacked velocity-over-pressure DOF vector) and $\delta = [\delta u; \delta p]$ the corresponding stacked increment; the Picard choice freezes $A = A(u_k)$ while full Newton adds the transport-velocity Jacobian $\mathcal{C}'(u_k)$ of §4, selected at runtime. A matrix-free GMRES variant with a block-diagonal Cahouet–Chabard preconditioner [8] is available for problem sizes above the dense-LU memory ceiling.

5.2 Block-Schur projection

The projection solver is a semi-implicit (IMEX) reformulation of the per-step saddle (16) in the spirit of the dual-splitting / generalized-Stokes schemes of [13, 19]. In the time-continuous splitting, the momentum equation is advanced with BDF q , the viscous and pressure terms are taken implicitly at t^{n+1} , and the nonlinear convective term is taken explicitly at the BDF-extrapolated velocity $u_* = \sum_{i=0}^{q-1} \alpha_i u^{n-i}$:

$$\begin{aligned} \frac{\gamma_0}{\Delta t} u^{n+1} - \nu \Delta u^{n+1} + \nabla p^{n+1} &= f_{\text{hist}} - \nabla \cdot (u_* \otimes u_*), \\ \nabla \cdot u^{n+1} &= 0, \end{aligned} \quad (18)$$

with $f_{\text{hist}} = \sum_i (\beta_i / \Delta t) u^{n-i}$ the BDF history. Equation (18) is a *generalized Stokes* (linear, velocity–pressure-coupled) problem at each step: the convective nonlinearity has been moved entirely to the known right-hand side, so the implicit operator is fixed in time. Discretizing (18) on the BDM $_k$ / P_{k-1} pair of §3.1 with the SIPG viscous form of (10) and the element-local divergence/gradient operators D, G of §3.3 reproduces the saddle (16) but with the convective block $\mathcal{C}(u_*) u_*$ on the right-hand side rather than in the matrix. We solve this fixed saddle by an exact block-Schur (pressure-correction) factorization, implemented as a per-step splitting.

When convection is treated explicitly through u_* , (16) admits a block-Schur factorization that we implement as a per-step splitting. Defining the velocity Helmholtz block $H = (\gamma_0 / \Delta t) M + \nu a^{\text{SIPG}}$ and the pressure-Schur complement $S_H = D H^{-1} G - \varepsilon I$, the per-step update $(u^n, u^{n-1}, \dots) \mapsto (u^{n+1}, p^{n+1})$ proceeds through three substeps:

1. *Velocity extrapolation and residual at the linearization point.* Form $u_* = \sum_{i=0}^{q-1} \alpha_i u^{n-i}$ with $(\alpha_0) = (1)$ for BDF1 and $(\alpha_0, \alpha_1) = (2, -1)$ for BDF2, and evaluate the full saddle residual of (16) at $(u_*, 0)$,

$$F_u = H u_* + \mathcal{C}(u_*) u_* - M f_{\text{hist}} - \text{lift}_{\text{BC}}, \quad F_p = -D u_*. \quad (19)$$

The Lax–Friedrichs convective contribution $\mathcal{C}(u_*) u_*$ already carries the boundary-face treatment of §3.3: Dirichlet faces inject the prescribed inflow u_{BC} as the neighbor trace, free-stress faces use $u^+ = u^-$, so no further surface terms are added at the splitting step.

2. *Pressure-Schur solve.* The interior block-LU of (16) gives

$$S_H p^{n+1} = D H^{-1}(-F_u) - F_p. \quad (20)$$

On closed domains the discrete pressure-Schur has a one-dimensional constant nullspace; we lift it with the Brezzi–Pitkäranta [7] regularizer εI , $\varepsilon \sim 10^{-10}$, which is consistent to roundoff with mean-projection and avoids the need to caller-supply a deflation vector. The dense factor of S_H is of size $\dim S_H = |\mathcal{T}_h| \cdot k(k+1)/2$ (element-local P_{k-1} pressure modes) and is reused across all BDF steps.

3. *Helmholtz velocity correction.*

$$H \delta u = -F_u - G p^{n+1}, \quad u^{n+1} = u_* + \delta u. \quad (21)$$

The strong Dirichlet velocity BC is imposed on H by *interior reduction*: BC rows of the dense H are replaced by the identity and BC columns are zeroed before LU factorization, so that δu vanishes on Dirichlet DOFs and u_* already carries the prescribed boundary trace through the lift in (19). Free-stress faces enter H through the SIPG bilinear form of (10) by setting the boundary face contributions to zero (the natural BC is absorbed by integration by parts).

With convection explicit, H and S_H depend only on $(\Delta t, \nu, k, \text{mesh})$ and can be dense-LU factored once at construction; per step cost is then two LU back-solves plus one matrix-free evaluation of $D H^{-1}(\cdot)$. The scheme is the classical pressure-correction factorization [8] applied to the BDM $_k$ / P_{k-1} pair; the implementation contribution here is the BDM-specific factorization in which D is element-local and the Schur factor is small. The scheme is semi-implicit: the Helmholtz block H carries only the mass and viscous terms, while convection is treated explicitly through the extrapolated residual $\mathcal{C}(u_*) u_*$ on the right-hand side. This is what allows H and S_H to be factored once and reused; it also means the per-step update is *not* algebraically identical to a Picard iteration of the fully-implicit monolithic (which carries $\mathcal{C}(u_*)$ in its left-hand side); the two coincide only in the Stokes limit where $\mathcal{C} = 0$. We verify in §8.6 that the block-Schur elimination is an exact factorization of the projection’s own saddle to LU roundoff, and that the explicit convective treatment imposes a CFL-type step limit (§8.7).

A practical limitation of the current implementation is that the pressure step uses the closed-domain Brezzi–Pitkäranta regularizer uniformly; the consistent mixed-BC pressure treatment of [13], a SIPG- P_{k-1} Laplacian with outflow Dirichlet $p = 0$ together with the viscous curl–curl contribution on Dirichlet walls, is not yet wired through. Inflow/outflow runs (§8.4, §9.5) therefore rely on the ε -stabilized closed-domain Schur with outflow handled implicitly through the $(\nu \nabla u - pI) \cdot n = 0$ natural condition in the viscous block; this works at the modest Reynolds numbers considered here, and we revisit it in §10.

Boundary conditions. The boundary treatment is the subtle part of the splitting and we state it in full. Velocity Dirichlet data on Γ_D (inflow, walls, the cylinder) enters in two places that must be kept consistent. First, the SIPG symmetry and penalty terms of (10) contribute a boundary lift lift_{BC} to the momentum residual (19); this is the *weak* imposition that the viscous operator sees. Second, the prescribed trace must be injected *strongly* into the extrapolated linearization point on the Dirichlet DOFs, $u_* \leftarrow u_{\text{BC}}$ on Γ_D , before the residual (19) is formed. The interior reduction of substep 3 makes δu vanish on those DOFs, so the boundary trace of $u^{n+1} = u_* + \delta u$ is carried entirely by u_* ; omitting the injection leaves $u_*[\Gamma_D] = 0$ from a rest start and the inflow is never seen by the velocity update (the discrete solution then decays to a spurious zero-flow state). This mirrors the monolithic Newton path, which initializes each outer iterate with $u_k = u_{\text{BC}}$ on Γ_D ; a regression test (§8.6) asserts that one projection step reproduces the prescribed boundary velocity to LU roundoff. Outflow (Γ_N) is the natural free-stress condition $(\nu \nabla u - pI) \cdot n = 0$: those faces are flagged and their SIPG and Lax–Friedrichs surface contributions are dropped, so the condition is imposed weakly through integration by parts and requires no explicit data. Finally, because convection is explicit, an impulsive start excites the CFL bound of §8.7; we therefore ramp the inflow amplitude over a short interval $[0, t_{\text{ramp}}]$ by $u_{\text{BC}}(t) = r(t) u_{\text{BC}}$ with the raised-cosine factor

$$r(t) = \begin{cases} \frac{1}{2}(1 - \cos(\pi t/t_{\text{ramp}})), & 0 \leq t < t_{\text{ramp}}, \\ 1, & t \geq t_{\text{ramp}}, \end{cases} \quad (22)$$

rising smoothly from $r(0) = 0$ to $r(t_{\text{ramp}}) = 1$ (here $t_{\text{ramp}} = 0.5$), which the BDF history then absorbs.

Discretization of the projection operators. The four operators in the factorization inherit the BDM_k/P_{k-1} structure of §3.1. The Helmholtz block $H = (\gamma_0/\Delta t)M + \nu a^{\text{SIPG}}$ uses the element-local BDM mass matrix M (block-diagonal per element under the Piola map) and the $\text{H}(\text{div})$ -conforming SIPG form (10), which acts only on the tangential trace. The divergence $D: \text{BDM}_k \rightarrow P_{k-1}$ and gradient $G = -D^\top$ (the negative adjoint of §3.1) are exact, element-local cross-space operators (the discrete divergence of a BDM_k field lies in P_{k-1} pointwise), so D and G are block-diagonal and cheap. The pressure Schur complement $S_H = D H^{-1} G - \varepsilon I$ is a discrete pressure-Poisson operator on the P_{k-1} space of dimension $|\mathcal{T}_h| \cdot k(k+1)/2$; it is the only globally-coupled factor and is assembled column-by-column through H^{-1} once and dense-LU factored. Because the $\text{H}(\text{div})$ -conforming velocity space makes Du^{n+1} vanish pointwise rather than in a weak sense, the discrete incompressibility is satisfied to solver tolerance ($\|\nabla \cdot u\| \lesssim 10^{-9}$ in §8.4) independent of the splitting.

6 Differentiable formulation

The forward solver of §2–§5 is implemented so that every quantity computable from input data (mesh geometry parameters, initial and boundary conditions, viscosity, solver hyperparameters) can be obtained as a smooth function with a machine-precision derivative through reverse-mode AD. This section describes the structural elements that this requires.

6.1 Differentiation through implicit time-stepping

Each BDF q step is the solution of a linear (Picard linearization) or nonlinear (Newton) system in the state (u^{n+1}, p^{n+1}) . For the Picard path, $(u^{n+1}, p^{n+1}) = A(u^*; \theta)^{-1} b(\theta, u^n)$ is differentiated through the standard linear-solve sensitivity formula; the AD framework provides this through a custom vector–Jacobian-product on the dense LU back-solve that reuses the cached factor in the backward pass. The cost of one backward pass is one transpose triangular solve per forward solve, comparable to the forward back-solve.

For the Newton path, an outer iteration $u_{k+1} = u_k - J^{-1}F$, with $J = \partial F/\partial u$ the Newton Jacobian of the residual F , has a differentiable implicit map at the converged root via the implicit-function theorem: writing θ for the vector of design parameters (mesh geometry, boundary/initial data, solver hyperparameters) and u^* for the converged solution (the root, not the transport velocity u_* of §4), at $F(u^*; \theta) = 0$ one has $\partial u^*/\partial \theta = -J^{-1} \partial F/\partial \theta$. The framework’s support for custom implicit-function differentiation expresses this directly. In our experiments we use a fixed-iteration unrolled Newton (typically one Picard iteration per BDF step in transient runs); the unrolled trace gives the gradient through the same arithmetic without explicit invocation of the implicit-function theorem.

6.2 Non-smoothness in the Lax–Friedrichs flux

The convective bilinear form (12) contains the dissipation speed $\lambda = \max(|u_* \cdot n^-|, |u_* \cdot n^+|)$, which is non-differentiable at $u_* \cdot n = 0$ (corresponding to streamlines tangent to a face) and at the maximum-switching point. We adopt the standard elementwise subgradient convention $\partial|x|/\partial x = \text{sign}(x)$ with $\text{sign}(0) = 0$. The resulting Clarke subgradient has zero Lebesgue measure on the flow, and we have not found it to cause optimizer pathologies in any of the applications below. The hand-derived Newton extra term contains a corresponding $\text{sign}(u_* \cdot n)$ contribution; we have verified that the reverse-mode JVP through the convective form agrees with the analytic hand-derivation to LU roundoff on the cylinder mesh.

6.3 Differentiable mesh construction

The mesh constructor for body-fitted geometries takes scalar geometric parameters

$$\theta_{\text{geom}} = (C_x, C_y, R, R_{\text{outer}}, s_r, \dots), \quad (23)$$

namely the cylinder-center coordinates (C_x, C_y) , the cylinder radius R , the outer bounding-box half-width R_{outer} , the radial growth ratio s_r of successive annular layers, and optional Fourier-mode coefficients for non-circular shapes (introduced in §9.4), and returns the vertex array, triangle connectivity, and curved-element node placement. The constructor is written end-to-end in the AD-traceable array language: the conforming

triangulation of the bounding square around the cylinder, the radial annular layers, and the boundary-blend node placement that maps the inner ring to $r = R$ are all differentiable functions of θ_{geom} . Consequently any downstream quantity computed from a forward simulation on this mesh has a smooth derivative with respect to θ_{geom} .

Two notes on this construction. *Grid-alignment constraint*: the constructor enforces that the bounding square of the cylinder has integer-indexed corners in the quadrilateral substructure of the outer mesh. This constraint restricts the admissible *discrete* values of C_y to a finite set (multiples of the background-grid spacing $h_y = (Y_{\text{max}} - Y_{\text{min}})/K_y$, with $[Y_{\text{min}}, Y_{\text{max}}]$ the vertical domain extent and K_y the number of background cells in y), which is why finite-difference-on-rebuild gradient probes with a step $\delta < h_y$ can fail to move from one aligned C_y to the next. The AD path does not see this constraint: it differentiates the constructor as a continuous function of C_y , and the alignment constraint affects only whether the resulting mesh is conforming, not whether the gradient is well-defined. *Curved-element cubature*: the curved annular ring requires the conical cubature [26] for the mass and SIPG bilinear forms; nodal quadrature underestimates the Jacobian variation over curved elements and reduces the formal convergence rate from h^{k+1} to h^1 .

7 Implementation

The framework is implemented in JAX [5], a numerical computing library that provides reverse-mode AD, just-in-time compilation to XLA, and a unified array API targeting CPU, GPU, and TPU backends. We use JAX rather than alternatives (PyTorch, Julia/Zygote, TensorFlow) for three reasons. First, its function-transformation model (just-in-time compilation, vectorized mapping, reverse-mode differentiation, and custom vector–Jacobian products) composes cleanly with the operator-level structure of a finite-element solver: a single function returning a residual or a per-step state-update map becomes both forward and backward by transformation, without duplication of the source. Second, the XLA backend provides ahead-of-time compilation of the BDF step into one fused kernel, which is necessary for sustained throughput on GPU and for the viability of the JIT-compiled per-step cost we report in §8.8. Third, the same source runs on CPU and on CUDA GPU hardware without modification, which has been useful for the development cycle: small-mesh sentinels on a laptop CPU, production runs on remote Google Cloud (GCP) instances. All CPU wall-clock numbers reported in this paper were measured on a GCP `c4-highmem-16` instance (Intel Xeon Platinum 8581C at 2.30 GHz; 16 vCPUs over 2 sockets of 8 physical cores, one thread per core; 121 GiB RAM); the GPU portability was exercised on NVIDIA L4 and V100 accelerators.

Compilation strategy. The BDF step function is just-in-time compiled once with u^* as an explicit tracer argument; the inner GMRES, the matvec, and the preconditioner stack therefore trace only once per BDF order rather than once per BDF step. The dense LU factors of H and S_H in the projection variant of the solver (§5.2) are built once at construction time and reused for every step. The same factor is used in the forward and backward passes, with no separate transpose-factor or adjoint matrix.

Test infrastructure. The verification testbed of §8 is also the regression suite: every benchmark and unit test runs on a single CPU thread in under a minute, so the full suite executes on each change to the code. Each sentinel is designed to pin one falsifiable property of a scheme. Manufactured-solution tests check that every scheme and order attains its design rate (spatial h^{k+1} , temporal Δt^q); the pressure-robustness sentinel (§8.1) checks that the $H(\text{div})$ pair holds machine-zero velocity error where a nodal pair cannot; a block-Schur factorization test checks that the projection reproduces the monolithic saddle solve to LU roundoff (§8.6); a Dirichlet-trace regression checks the boundary-velocity injection; an L^2 -stability test marches the bare scalar-advection operator under a strained divergence-free flow, which a correctly upwinded operator cannot fail; and the reverse-mode gradient is checked against finite differences (§6).

Property-level sentinels matter here for two reasons. The three solvers share operators (the same convective flux, viscous bilinear form, and mass apply), so a change in one place can silently break another; a test that exercises each property in isolation localizes a regression to the operator at fault rather than to a downstream symptom, and catches defects that coarser end-to-end checks (which can be insensitive to a sign or a scaling) pass over. Because every convergence rate and error floor reported in §8 is regenerated by running the suite from the same source, the claims are reproducible rather than asserted. The framework, the testbed, and the application demonstrations of §9 are released as a single open-source repository, `ddg`.

8 Numerical verification

We verify the forward solver against the standard benchmark suite for incompressible DG-INS. The objective is to establish that the spatial and temporal discretization behaves as expected; the differentiable applications in §9 take this verification as a given.

8.1 Stokes pressure-robustness sentinel

The pressure-robustness sentinel [23] tests whether a discrete inf-sup-stable pair reproduces an analytic Stokes solution when the velocity space is manufactured to be div-free. The BDM_k/P_{k-1} pair reaches a velocity-error floor at the ε pressure-nullspace regularization (machine zero at $\varepsilon = 0$, up to LU roundoff). An equal-order Lagrangian-DG pair with Brezzi–Pitkäranta stabilization, by contrast, floors at a level proportional to $\|\nabla p\|_{L^2}$, the textbook pressure-robustness failure mode of LBB-stabilized equal-order discretizations [17]. The leftmost panel of Figure 2 shows the result: the H(div)-conforming BDM pair holds at machine-zero L^2 velocity error independently of the spatial mesh, while the equal-order and mixed-order Lagrangian pairs deteriorate as h shrinks because the LBB stabilization amplifies the artificial ∇p contribution.

8.2 Kovasznay convergence

The Kovasznay analytical solution at $\text{Re} = 40$ on the slice $[-0.5, 1] \times [-0.5, 1.5]$ has closed-form velocity and pressure. The middle panel of Figure 2 shows the L^2 velocity error versus h for BDM_1 , BDM_2 , and BDM_3 ; the observed rates approach the expected h^{k+1} slope (BDM_3 still pre-asymptotic at the finest mesh shown). (The steady Kovasznay solve uses the Newton linearization; the frozen-coefficient Picard iteration does not converge at this Reynolds number.) Because the block-Schur projection treats convection explicitly, its spatial and temporal orders are verified separately and are reported in §8.6.

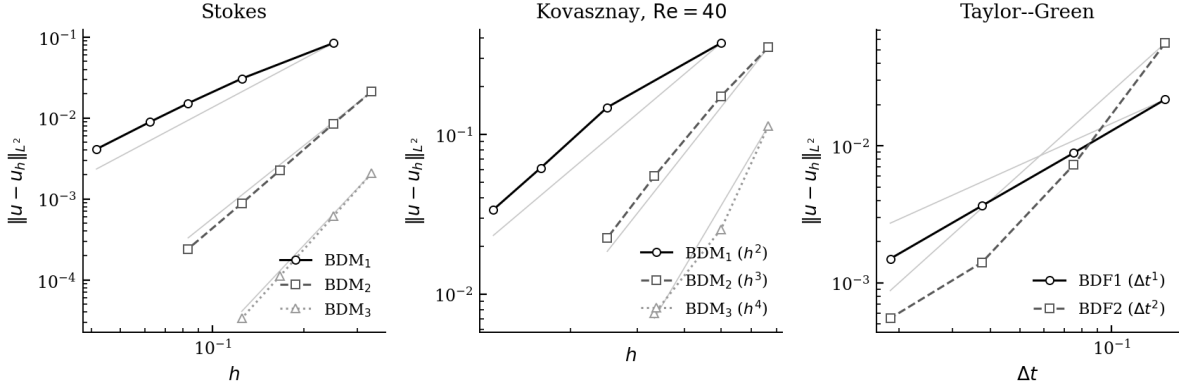


Figure 2: Benchmark convergence. Left: Stokes velocity-error decay at fixed $\text{Re} = 0$; the BDM_k/P_{k-1} pair holds at the machine-zero pressure-robustness floor independently of h . Middle: Kovasznay velocity-error decay at $\text{Re} = 40$ approaches the expected h^{k+1} rate (BDM_3 pre-asymptotic at the finest h shown). Right: Taylor–Green temporal refinement (BDF1, BDF2) on a fixed BDM_2 mesh; the design orders 1 and 2 are recovered cleanly in the self-convergence study (§8.6).

8.3 Ghia lid-driven cavity

Table 1 compares the monolithic Newton and the block-Schur projection against the Ghia 1982 centerline tables at $\text{Re} \in \{100, 400\}$. The two solvers agree to all reported digits where the projection is stable; §8.7 discusses the configuration where it is not.

8.4 Schäfer–Turek cylinder benchmark

The Schäfer–Turek 2D-2 benchmark at $\text{Re} = 100$ measures the mean drag coefficient $\langle C_D \rangle = \langle F_x \rangle / (\frac{1}{2} \rho U_{\text{mean}}^2 d)$, where $\langle \cdot \rangle$ denotes the time average $\langle \phi \rangle = |I|^{-1} \int_I \phi dt$ over a developed-flow window I (a one-argument

Table 1: Lid-driven-cavity verification. Ghia 1982 centerline u-velocity RMSE under both solver paths on the same mesh, Δt , and tolerance. Here n denotes the number of cells per side of the structured cavity mesh.

configuration	monolithic Ghia RMSE	projection Ghia RMSE
Re = 100, BDM ₁ , $n = 8$	1.303×10^{-2}	1.303×10^{-2}
Re = 100, BDM ₂ , $n = 8$	8.045×10^{-3}	8.045×10^{-3}
Re = 400, BDM ₁ , $n = 12$	4.389×10^{-2}	4.389×10^{-2}
Re = 400, BDM ₂ , $n = 12$, $\Delta t = 0.05$	4.671×10^{-3}	diverged
Re = 400, BDM ₂ , $n = 12$, $\Delta t = 0.025$	4.672×10^{-3}	4.673×10^{-3}

mean, not to be confused with the two-argument face inner product $\langle \cdot, \cdot \rangle_\gamma$ of §3.1), F_x and F_y are the streamwise and transverse components of the hydrodynamic force on the cylinder, ρ the (constant) fluid density, and d the cylinder diameter; the peak lift amplitude $\hat{C}_L$ is the maximum over time of the lift coefficient $C_L = F_y / (\frac{1}{2} \rho U_{\text{mean}}^2 d)$ (same normalization), the Strouhal number is $\text{St} = f d / U_{\text{mean}}$ (f the vortex-shedding frequency, U_{mean} the mean inflow speed), and the pressure difference is $\Delta p = p_{\text{front}} - p_{\text{back}}$ across the cylinder, with reference values $\langle C_D \rangle = 3.22$, $\hat{C}_L \approx 1.0$, $\text{St} \approx 0.30$, and $\Delta p = 2.48$. On our near-canonical setup (channel 2.2×0.40 , cylinder at $C_y = 0.20$, $|\mathcal{T}_h| = 736$, BDM₃, BDF2 with four Newton iterations per step, $\Delta t = 0.02$, $T = 5$) the solver reaches $\langle C_D \rangle = 3.52$, $C_L \in [-1.06, 0.91]$, $\text{St} \approx 0.26$, $\Delta p \in [2.30, 3.22]$. The divergence stays $\|\nabla \cdot u\| \lesssim 10^{-8}$ throughout, settling to $\sim 10^{-9}$ over the developed window. The $\sim 10\%$ departure of $\langle C_D \rangle$ from the canonical reference is consistent with the mesh-aspect-ratio mismatch between our channel (2.2×0.40) and the canonical 2.2×0.41 , and with the lower-than-canonical resolution. The result validates that the von Kármán shedding transition is reproduced and the divergence-free property holds; it is not a benchmark-precision match. The diagnostics (Figure 3) show the ramp, the exponential amplification of the lift instability, and the limit-cycle saturation; Figure 4 renders the vorticity and speed fields of a companion BDM₂ run showing the developed von Kármán street.

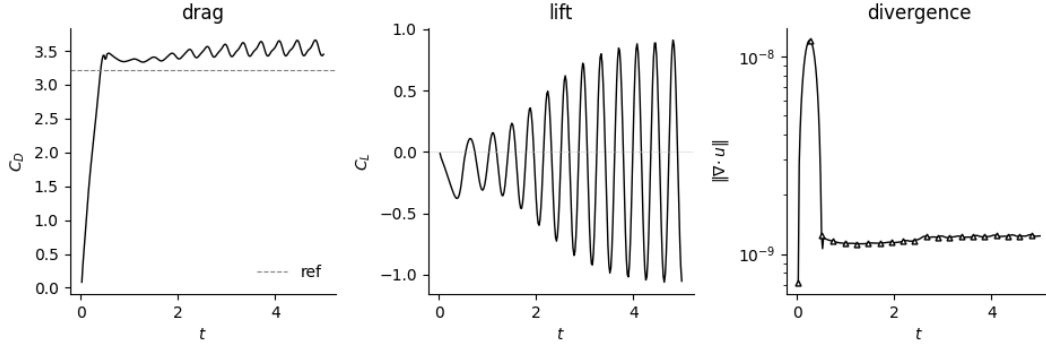


Figure 3: Schäfer–Turek 2D-2 cylinder benchmark, BDM₃, $|\mathcal{T}_h| = 736$, $\Delta t = 0.02$, $T = 5$. Top: C_D trajectory with the Schäfer–Turek reference $\langle C_D \rangle = 3.22$ marked. Middle: C_L , with the exponential growth of the wake instability visible between $t \in [1, 3]$ before limit-cycle saturation. Bottom: discrete divergence norm, $\|\nabla \cdot u\| \lesssim 10^{-8}$ throughout, settling to $\sim 10^{-9}$.

8.5 Boussinesq rising bubble

A buoyancy-driven rising bubble couples the momentum equation (1) to a passive-scalar transport equation $\partial_t c + u \cdot \nabla c = \kappa \Delta c$, where c is the transported scalar concentration and $\kappa > 0$ its diffusivity. The scalar enters the momentum equation as a body force $f_{\text{buoy}} = -\beta_b g c \hat{e}_y$, with β_b a (dimensionless) buoyancy coefficient (unrelated to the saddle operator A of §4), $g > 0$ the gravitational acceleration, and $\hat{e}_y$ the upward unit vector. The bubble is the lighter fluid, marked by a negative cosine bump ($c < 0$ inside, zero outside), so the buoyancy force points upward there and the bubble rises. The configuration follows a standard buoyant-bubble setup on a $[0, 1] \times [0, 2]$ column with the bubble of radius 0.25 centered at $(0.5, 0.7)$. The scheme is sequentially

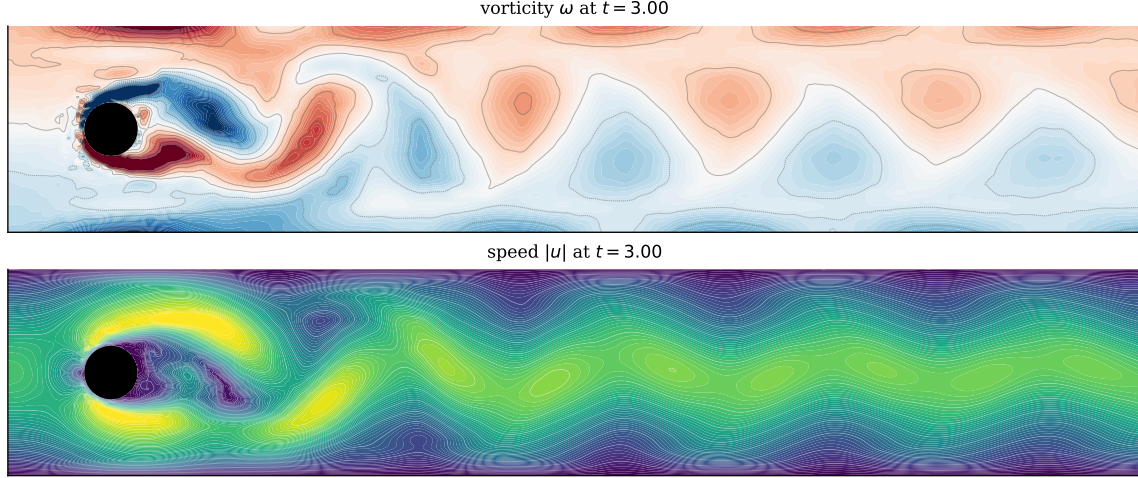


Figure 4: Developed von Kármán street on the Schäfer–Turek $\text{Re} = 100$ cylinder (monolithic coupled-Newton BDM_2 , $|\mathcal{T}_h| = 768$, $\Delta t = 0.02$; a small one-sided inflow perturbation seeds the shedding on the centered-cylinder geometry). Top: vorticity ω ; bottom: speed $|u|$. Both fields are post-processed with shared-node averaging and cubic interpolation onto a fine grid for a continuous (non-faceted) DG visualization. The alternating shed vortices convect the full channel length and the discrete divergence stays at $\|\nabla \cdot u\| \lesssim 10^{-9}$ throughout.

coupled (one scalar step, then one velocity step, per time level). Figure 5 shows the scalar at four times spanning the rise: the bubble deforms into a mushroom cap whose edges roll up into two counter-rotating filaments. The velocity is pointwise divergence-free throughout ($\|\nabla \cdot u\| \lesssim 5 \times 10^{-11}$), the buoyancy entering as a body force rather than as a perturbation to the divergence constraint. The block-Schur projection and monolithic coupled-Newton solvers produce the same rise; their scalar fields differ by a relative L^2 norm of 5×10^{-5} at $t = 2$, consistent with their explicit and implicit convective treatments (§8.6).

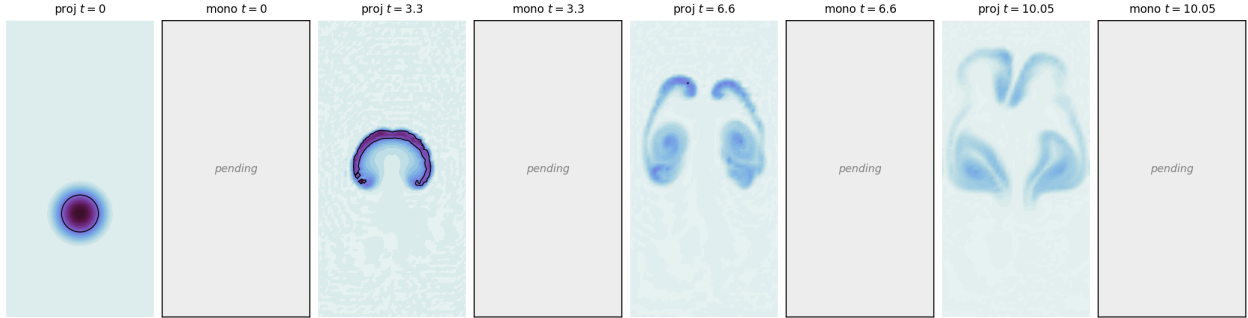


Figure 5: Boussinesq rising bubble (BDM_3 , $\kappa = 10^{-4}$): the scalar c at four times $t \in [0, 10]$, each shown as an adjacent projection/monolithic pair (left: block-Schur projection; right: monolithic coupled-Newton). The bubble forms a mushroom cap that rolls up into two counter-rotating filaments; the interface is the c midline contour. Both solvers hold the velocity pointwise divergence-free ($\|\nabla \cdot u\| \lesssim 5 \times 10^{-11}$); a matched-resolution comparison (§8.6) finds the two scalar fields agree to a relative L^2 of 5×10^{-5} .

8.6 Projection vs. monolithic head-to-head

A unit-test sentinel assembles the projection’s own saddle $[H \ G; -D \ -\varepsilon I]$ on a small Kovasznay slice with a non-zero linearization point and solves it via both a dense LU and the block-Schur elimination. The two agree to LU roundoff ($\|u_{\text{proj}} - u_{\text{ref}}\|/\|u_{\text{ref}}\| < 10^{-9}$), confirming the factorization is exact. This is the projection’s *own* (convection-free) saddle: because the scheme treats convection explicitly, a single projection

step is not bit-identical to a Picard iteration of the fully-implicit monolithic except in the Stokes limit $\mathcal{C} = 0$. A companion regression test starts from rest with a non-zero Dirichlet BC and asserts that one projection step injects the prescribed boundary velocity onto the Dirichlet DOFs to LU roundoff; this is the test that surfaced the BC-injection bug discussed in §5.2.

Order of accuracy. The projection’s spatial and temporal orders are verified against exact solutions. *Spatial:* on the steady Stokes problem, where the explicit-convection treatment is inactive and the block-Schur is an exact factorization of the resulting saddle, the projection’s L^2 velocity error decays at the optimal h^{k+1} rate, reproducing the monolithic Newton solve’s asymptotic slopes (1.92, 3.31, 4.24 for $k = 1, 2, 3$; expected 2, 3, 4), with error magnitudes agreeing to within $\sim 10\%$ on the same meshes and narrowing under refinement. *Temporal:* on the unsteady Taylor–Green vortex at $\text{Re} = 100$, refining Δt on a fixed BDM₃ mesh and isolating the time-discretization error by self-convergence (measuring each Δt against the finest- Δt solution on the same mesh, so the constant spatial error cancels) recovers the design BDF orders: slope ≈ 1 for BDF1 and ≈ 2 for BDF2. With the convective term assembled consistently, the projection reproduces the monolithic solution on convective flows: on this Taylor–Green march the projection and monolithic final-time L^2 errors agree to 6.6×10^{-4} , the common spatial-discretization floor.

At the integration level we run both solvers on the same Schäfer–Turek $\text{Re} = 100$ cylinder (BDM₂, $|\mathcal{T}_h| = 768$, one-sided inflow perturbation to seed the instability) and compare the developed wakes (Figure 6). The semi-implicit projection reproduces the fully-implicit coupled solver’s von Kármán street: both reach a periodic vortex-shedding limit cycle with the same staggered spacing, and the projection holds the discrete divergence at $\|\nabla \cdot u\| \lesssim 10^{-9}$ throughout its $T = 8$ trajectory; the H(div)-conformity is preserved by the splitting. The two solvers are not expected to agree pointwise per step: they use different convective treatments (explicit-extrapolated vs. implicit) and, as a consequence, different admissible time steps (§8.7); a per-step velocity-difference norm is therefore not a meaningful accuracy measure between them, and we instead verify agreement at the level of the developed wake (Figure 6) and the order-of-accuracy study of §8.6. On the patched code, the projection step costs 53.9 ms per step on this BDM₂ cylinder ($|\mathcal{T}_h| = 768$, $n_u = 5940$, single CPU thread, averaged over the developed-wake portion of the $T = 8$ trajectory); the per-step speedup over the monolithic solver is quantified on matched cavity configurations in the scaling sweep of §8.8 (Table 2), where it grows with mesh size and order and reaches $\sim 21\times$ on the $\text{Re} = 400$ step, set by the projection’s factored-once-reused operators versus the monolithic’s per-Newton re-factorization.

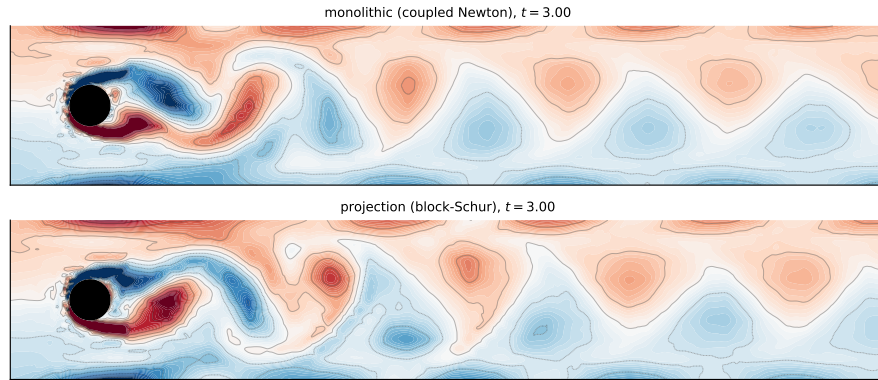


Figure 6: Monolithic coupled-Newton (top) vs. block-Schur projection (bottom) vorticity on the Schäfer–Turek $\text{Re} = 100$ cylinder, compared at the *same* physical time $t = 3.0$. Both solvers run on the *identical* body-fitted O-grid (BDM₂, background grid 44×8 , $n_{\text{radial}} = 3$, $|\mathcal{T}_h| = 768$, $n_u = 5940$, $n_p = 2304$); only the solver and time step differ. The semi-implicit projection ($\Delta t = 5 \times 10^{-4}$) reproduces the von Kármán street of the fully-implicit monolithic ($\Delta t = 0.02$) at the matched instant; the smaller projection step is forced by the explicit-convection CFL bound (§8.7), the one admissible difference between the two runs. The projection holds the discrete divergence at $\|\nabla \cdot u\| \lesssim 1.6 \times 10^{-9}$ throughout its $T = 8$ trajectory.

8.7 IMEX explicit-convection stability characterization

The projection variant’s explicit treatment of convection imposes a CFL-type upper bound on Δt that the monolithic Newton does not have. We characterize this on the $\text{Re} = 400$ BDM₂ lid-driven cavity with $n = 12$ cells per side, reporting the cell-Courant number $\text{CFL} = U_{\text{lid}} \Delta t / h_K$ (with $U_{\text{lid}} = 1$ the lid speed and h_K the near-lid element diameter). At $\Delta t = 0.05$ ($\text{CFL} \approx 0.6$ at the lid) the projection diverges: the velocity norm goes to infinity, divergence is lost, and the LU back-solve returns NaN. At $\Delta t = 0.025$ ($\text{CFL} \approx 0.3$) the projection recovers stability and matches the monolithic to three significant figures in Ghia RMSE (Table 1, last two rows). On the corrected solver the projection reproduces the monolithic Ghia RMSE to all reported digits wherever it is stable (Table 1: 1.303×10^{-2} , 8.045×10^{-3} , 4.389×10^{-2} at the three stable configurations); only this $\text{Re} = 400$ BDM₂ case at the larger step exceeds the explicit-convection CFL bound and diverges. A finer bisection of the stability boundary is feasible but not done here. The principled remedy is an implicit treatment of convection inside the projection step’s velocity solve (a Newton inner iteration on the explicitly-extrapolated convective term); we have prototyped this but defer its full characterization to a follow-up.

8.8 Per-step performance scaling

Table 2 reports the per-step wall-clock of the BDF2 lid-driven-cavity step on a single CPU thread of the `c4-highmem-16` instance of §7 for $n \in \{6, 10, 14\}$ at BDM₁ and BDM₂, comparing one monolithic linear solve (a single Picard linearization with dense LU) against one projection step. At the largest mesh in the sweep the projection runs $4.07\times$ faster than the monolithic at BDM₁ and $5.54\times$ at BDM₂; on the stable $\text{Re} = 400$ cavity step (Table 1) the ratio reaches $\sim 21\times$ at BDM₁ (monolithic 181.2 ms vs. projection 8.7 ms), where the projection matches the monolithic Ghia RMSE to all reported digits. The growth of the speedup with K and k reflects the cubic scaling of the monolithic LU’s $O((n_u + n_p)^3)$ per-step factorization cost, versus the projection’s smaller block factors built once and reused across every step.

Table 2: Scaling sweep on the lid-driven-cavity BDF2 step (ms / step, single CPU thread). Here n is the number of cells per side of the structured cavity mesh ($|\mathcal{T}_h| = 2n^2$ triangles) and $n_u = \dim V_h$ the velocity-DOF count.

	n	$ \mathcal{T}_h $	n_u	monolithic	projection	speedup
BDM ₁	6	72	240	144.3	104.5	$1.38\times$
	10	200	640	244.4	108.3	$2.26\times$
	14	392	1232	417.0	102.4	$4.07\times$
BDM ₂	6	72	576	282.3	164.6	$1.72\times$
	10	200	1560	516.1	182.8	$2.82\times$
	14	392	3024	1224.8	221.1	$5.54\times$

8.9 Stability and robustness

Table 3 summarizes which solver and test-case combinations we have driven to a good result. The rows are ordered as a *validation cascade*: the manufactured-solution and convergence tests at the top (spatial h^{k+1} , temporal Δt^q) establish that a scheme attains its design order before any steady benchmark or long-time application below it is attempted. This ordering is not incidental: a high-order scheme that does not converge at its design rate on the upper rows cannot be meaningfully assessed on the lower ones (a developed-flow picture from a scheme that is not provably high-order conveys nothing), so we do not expend the expensive long-time runs on a column until its convergence rows are established. We restrict the comparison to inf-sup-sound solvers: an LBB-unstable velocity–pressure pair is not a legitimate comparator (its results are meaningless however far down the cascade they appear to run), which is why no monolithic non-H(div) column is included. The H(div) monolithic (B) and block-Schur projection (C) clear the entire cascade (§8.6), including the buoyancy-driven rising bubble: both H(div) solvers march it into its developed regime, tracking each other to a relative 5×10^{-5} at $t = 2$, consistent with their differing implicit and explicit convective treatments, and reproducing the nodal penalty-projection (A) result.

The single $\times$ in Table 3 is the penalty-projection (A) on the pressure-robustness Stokes sentinel: its nodal velocity is not pointwise divergence-free, so the L^2 velocity error floors at a level proportional to $\|\nabla p\|$ and does not vanish as $h \rightarrow 0$ (§8.1, Figure 2), the failure mode the BDM pair is built to avoid, so the entry is an expected structural outcome rather than a solver defect. The $\text{Re} = 400$ cavity entry under (C) is a $\checkmark$ only within the explicit-convection CFL bound ($\Delta t \leq 0.025$, §8.7).

The three solvers compared. All three columns discretize the same incompressible Navier–Stokes system and a sequentially-coupled scalar transport where present; they differ in the velocity–pressure representation and in how the resulting algebraic system is advanced. **(A) Nodal penalty-projection.** A nodal (Lagrangian) equal-order DG velocity–pressure pair in which the pressure is recovered by a pressure-Poisson solve (a consistent splitting), so no saddle-point inf-sup condition is invoked; discrete stability is supplied instead by grad–div and continuity (face-jump) penalties in the style of [13], and convection is explicit (BDF-extrapolated). The velocity is only weakly (penalty-)divergence-free rather than pointwise, so the scheme is not pressure-robust. **(B) Monolithic BDM_k/P_{k−1} coupled Newton.** An H(div)-conforming BDM_k velocity with a discontinuous P_{k−1} pressure (Piola-mapped), a pointwise divergence-free, pressure-robust, inf-sup-stable pair; viscosity is treated by SIPG and convection by a Lax–Friedrichs flux on the tangential trace, and the full velocity–pressure saddle (with *implicit* convection) is solved monolithically by Newton: dense LU, or matrix-free GMRES under the block preconditioner of §5.2. **(C) Block-Schur projection on BDM_k.** The same BDM_k/P_{k−1} space and pointwise-divergence-free, pressure-robust properties as (B), but the saddle is solved by an exact block-Schur (pressure-correction) factorization with *explicit*-extrapolated (IMEX) convection (§5.2); its factored-once-and-reused operators make it markedly cheaper per step than the re-factorizing monolithic (up to $\sim 21\times$ on the configurations of §8.8) at the cost of an explicit-convection CFL limit (§8.7).

Table 3: Solver coverage across the validation cascade for the three solvers (A)–(C) of the text. $\checkmark$: a good result; $\times$: an expected structural failure; blank: not run or not established.

test case	(A) penalty-proj.	(B) monolithic BDM	(C) block-Schur proj.
Stokes (pressure-robust, h^{k+1})	$\times$	$\checkmark$	$\checkmark$
Kovaszny (steady, h^{k+1})	$\checkmark$	$\checkmark$	$\checkmark$
Taylor–Green (temporal Δt^q)	$\checkmark$	$\checkmark$	$\checkmark$
Lid-driven cavity $\text{Re} = 100$		$\checkmark$	$\checkmark$
Lid-driven cavity $\text{Re} = 400$		$\checkmark$	$\checkmark$
Schäfer–Turek cylinder $\text{Re} = 100$		$\checkmark$	$\checkmark$
Boussinesq rising bubble	$\checkmark$	$\checkmark$	$\checkmark$

9 Applications of differentiability

9.1 Automated tuning of numerical parameters

Several numerical parameters in the framework are conventionally set by hand, by physical scaling arguments, or by trial-and-error. We show that the differentiable forward solver makes them direct targets for stochastic-gradient optimization.

Joint $(\Delta t, \tau)$ optimization on the stability cliff. We optimize the loss

$$\mathcal{L}(\Delta t, \tau) = \text{barrier}(\|\nabla \cdot u\|_\infty) - \gamma_{\Delta t} \log \Delta t + w_{\text{KE}} (\langle \text{KE} \rangle_{t>T/2} - \text{KE}_{\text{ref}})^2, \quad (24)$$

on a fixed-shape BDF2 trajectory of the BDM₃ cavity at $\text{Re} = 10^4$. Here $\|\nabla \cdot u\|_\infty$ is the maximum over the trajectory of the pointwise discrete divergence; $\text{barrier}(x) = \beta^{-1} \text{softplus}(\beta(\log x - \log U_{\text{tgt}}))$, with $\text{softplus}(z) = \log(1 + e^z)$, a sharpness β and a target level U_{tgt} , is a soft one-sided barrier that grows sharply as the iterate approaches the explicit-convection CFL boundary (where the divergence blows up); $\text{KE}(t) = \frac{1}{2} \int_\Omega |u(\cdot, t)|^2 dx$ is the kinetic energy, $\langle \cdot \rangle_{t>T/2}$ the time average over the second half of the trajectory

(the operator of §8.4), and KE_{ref} a reference value from a stable baseline run; $\gamma_{\Delta t}, w_{\text{KE}} > 0$ are fixed scalar weights ($\gamma_{\Delta t} = 0.1$). The first term penalizes the divergence cliff, the second rewards larger Δt , and the third keeps the solution in the physical kinetic-energy regime. Adam on $(\log \Delta t, \log \tau)$ from the heuristic initial point $(\Delta t_0, \tau_0) = (0.50, 4.0)$, run for 90 steps, drives the loss from 7.97×10^{-2} to -4.08×10^{-2} at $(\Delta t^*, \tau^*) = (1.69, 2.40)$ (Figure 7). The longer trajectory is informative: Δt inflates by $3.38\times$, while τ *deflates* to $0.60\times$ its hand-tuned value; the two parameters move in opposite directions, exposing the fact that the heuristic τ was overdamped for the stable time step the optimizer actually wants to take. Past iteration ~ 90 the parameters continue to wander at the noise floor of the loss without delivering further improvement, so we truncate the displayed trajectory there. The trajectory is non-monotone: at iteration 10 Adam overshoots the cliff and the loss transiently rises to 24.6 before the gradient-clipped Adam recovers within several iterations. The overshoot makes the practical point that the implicit stability barrier, while sufficient as a soft penalty, does not eliminate the need for gradient clipping; with per-component gradient clipping at magnitude ≤ 0.1 the optimizer recovers cleanly. The optimizer locates the largest stable Δt without any explicit characterization of the stability boundary; it learns the cliff from the divergence signature alone.

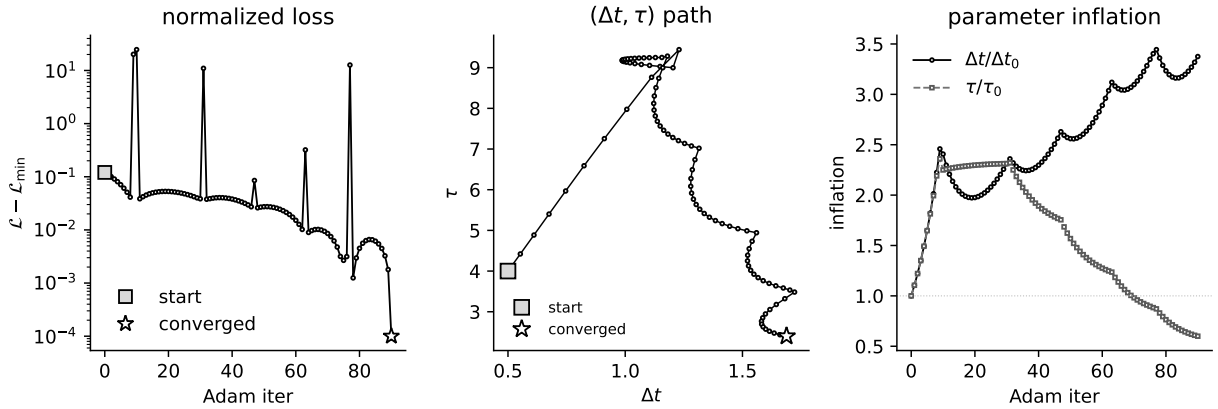


Figure 7: Joint $(\Delta t, \tau)$ optimization on the BDM₃ cavity at $\text{Re} = 10^4$, Adam through a fixed-shape BDF2 trajectory, 90 optimizer iterations (trajectory wanders at the noise floor afterwards). Left: loss trajectory (symlog scale); the optimizer overshoots the cliff at iteration 10 to a loss of 24.6 before recovering monotonically to a final value of -4.08×10^{-2} (star) from 7.97×10^{-2} (square). Middle: the joint $(\Delta t, \tau)$ trajectory. Right: Δt inflates $3.38\times$ over the heuristic baseline while τ deflates to $0.60\times$; the two parameters move in opposite directions, revealing that the heuristic τ was overdamped for the stable time step this discretization admits.

Learned multigrid smoother weights. The vertex-patch hp-multigrid V-cycle for the BDM_k viscous Helmholtz block has a per-level Jacobi relaxation weight w_ℓ that is conventionally set to a single scalar (typically $w_\ell = 1$ or $w_\ell = 2/3$ by heuristic). We make the w_ℓ Adam targets and backpropagate the squared CG residual after a fixed number of inner iterations through the V-cycle. Figure 8 shows the loss $\log_{10} \|r\|_2$, where r is the inner-CG residual vector after the fixed iteration budget and $\|\cdot\|_2$ its Euclidean norm, after 60 Adam iterations (the trajectory flatlines around iteration 30 and we truncate the display there). Adam drives the inner-CG residual norm from $10^{-1.04}$ at the heuristic w_ℓ down to $10^{-2.16}$ at the learned values, an asymptotic contraction-rate reduction from 0.354 (heuristic) to 0.116 (learned), which translates to $16 \rightarrow 9$ inner CG iterations to reach a fixed relative residual of 10^{-9} , a $1.8\times$ reduction in inner iteration count at no additional outer cost.

9.2 4D-VAR: formulation and 1D validation

The fourth problem class is variational data assimilation (4D-VAR): recover an unknown initial state from sparse spatiotemporal observations of the ensuing flow. Writing the discrete forward map as the trajectory operator $\mathcal{M}_{t_k}$ that advances the initial state u_0 to time t_k (a composition of the BDF/Newton steps), and

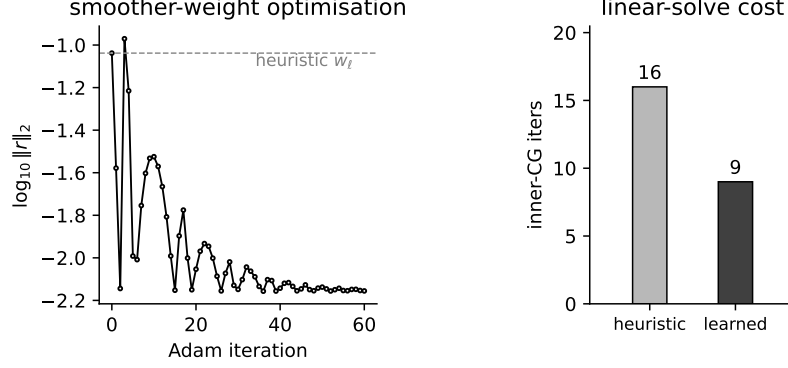


Figure 8: Learned per-patch Jacobi-smoother weights in the hp-multigrid V-cycle. Left: the loss $\log_{10} \|r\|_2$ after a fixed inner-CG budget over Adam iterations, dropping from -1.04 (heuristic w_ℓ) to -2.16 (learned). Right: the resulting linear-solve cost, the inner-CG iteration count to reach $\|r\|_2 \leq 10^{-9}$, which falls from 16 to 9 (a $1.8\times$ reduction), reflecting a contraction-rate reduction from 0.35 to 0.12.

$\mathcal{H}_k$ for the observation operator at t_k , 4D-VAR minimizes the regularized observation misfit

$$\mathcal{J}(u_0) = \frac{1}{2} \sum_{k=1}^K \|\mathcal{H}_k \mathcal{M}_{t_k}(u_0) - y_k\|^2 + \frac{\lambda_b}{2} \|u_0 - u_b\|^2, \quad (25)$$

with observations y_k , background (prior) state u_b , and Tikhonov weight λ_b . Its gradient,

$$\nabla \mathcal{J} = \sum_{k=1}^K (\partial \mathcal{M}_{t_k})^\top \mathcal{H}_k^\top (\mathcal{H}_k \mathcal{M}_{t_k}(u_0) - y_k) + \lambda_b (u_0 - u_b), \quad (26)$$

is the discrete adjoint: the transposed trajectory Jacobian $(\partial \mathcal{M}_{t_k})^\top$ is precisely what one reverse-mode sweep through the unrolled time stepping computes, so the gradient is obtained with no separately-derived adjoint model. The recovered initial state is the minimizer

$$u_0^* = \arg \min_{u_0} \mathcal{J}(u_0), \quad (27)$$

which we compute by Adam on the gradient (26).

1D twin-experiment validation. We first validate the machinery on a twin experiment with the one-dimensional viscous Burgers equation, where the recovery target is unambiguous. The forward model is

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [0, L], \quad t \in [0, T], \quad (28)$$

on the periodic domain of length $L = 2\pi$ with periodic boundary conditions, where ν is the kinematic viscosity (diffusion coefficient), set to $\nu = 0.02$, and $T = 1$ the final time. Equation (28) is discretized pseudo-spectrally (Fourier derivatives with 2/3 dealiasing on a uniform 128-point grid) and advanced with the classical explicit four-stage Runge–Kutta scheme at $\Delta t = 2 \times 10^{-3}$; the unknown is the periodic initial field $u_0(x)$. The AD gradient matches a central finite difference to a relative 10^{-10} , confirming the adjoint. With a well-posed observation budget (32 probes $\times$ 20 times, i.e. 640 observations for 128 unknowns) Adam drives the objective down five orders of magnitude and recovers the initial profile to a relative L^2 error of about 6%, the residual confined to the small scales that viscosity has already erased between the probes (Figure 9). Starving the budget to $4 \times 3 = 12$ observations reproduces the underdetermined regime: the data misfit still falls, but the recovered state is no longer constrained (relative error $\approx 40\%$). The same machinery applied to the two-dimensional cylinder is the subject of §9.3.

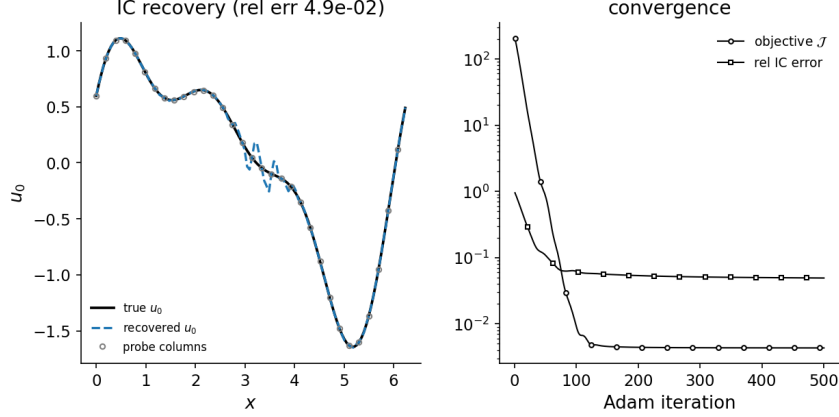


Figure 9: *Validation placeholder*. 1D viscous-Burgers 4D-VAR twin experiment. Left: the recovered initial condition (dashed) overlaid on the truth (solid), with the spatial probe columns marked; the two are visually indistinguishable apart from a small under-resolved wiggle. Right: the objective $\mathcal{J}$ and the relative initial-condition error against Adam iteration. The reverse-mode gradient agrees with a central finite difference to a relative 10^{-10} .

9.3 4D-VAR initial-condition recovery

We pose the inverse problem: given sparse downstream probe observations of velocity on a short observation window, recover the initial condition. The forward model is the BDM₂ smoke cylinder solver at $\text{Re} = 100$, $|\mathcal{T}_h| = 432$, $\Delta t = 0.05$; the truth u_0 is the state after four warm-up BDF2 steps from rest. The (linear) observation operator Π samples the velocity components (u_x, u_y) at six probe points along the downstream rake $y = C_y$, $x \in \{C_x + i\ell_p : i = 1, \dots, 6\}$ (with ℓ_p the probe spacing), at every step of an $n_{\text{obs}} = 4$ observation window. The total observation budget of 48 scalar values is heavily underdetermined relative to the 3,354-dimensional unknown.

The loss is

$$\mathcal{L}(u_0) = \sum_{s=1}^{n_{\text{obs}}} \|\Pi \mathcal{S}^s(u_0) - y_s\|_2^2 + \lambda_{\text{bg}} \|u_0\|_2^2, \quad (29)$$

where $\mathcal{S}^s(u_0)$ is the discrete state after s BDF2 steps from the initial condition u_0 (the solution map; not the Lax–Friedrichs flux $\mathcal{F}^{\text{LF}}$ or the face set $\mathcal{F}_h$), $y_s = \Pi \mathcal{S}^s(u_0^{\text{truth}})$ the data recorded at step s from the truth, $\|\cdot\|_2$ the Euclidean norm, and $\lambda_{\text{bg}} > 0$ a scalar weight; the second term is a background regularization toward $u_0 = 0$ (rest). The reverse-mode AD pass through the implicit BDF2 trajectory has cost equal to one forward pass, independent of the 3,354-dimensional parameter count. Figure 10 shows the trajectory. Adam at learning rate $\eta = 5 \times 10^{-4}$ with per-component gradient clipping at magnitude ≤ 0.1 drives the observation loss down into the $\sim 10^{-3}$ envelope over 15 iterations with non-monotone bouncing; the relative recovered-IC distance $\|u_0^{\text{rec}} - u_0^{\text{truth}}\|/\|u_0^{\text{truth}}\|$ stays nearly flat near 2.1, the manifold floor consistent with the under-determined-ness of the inverse problem at this observation budget (48 scalars against 3,354 unknowns). The figure demonstrates that the reverse-mode AD pass through the implicit BDF2 trajectory delivers usable gradients; recovering the initial condition itself requires a more informative observation operator.

No hand-coded tangent-linear or adjoint model is required for this demonstration; the implicit-function differentiation of the BDF inner solve (§6.1) is handled by the framework’s custom vector–Jacobian-product on the dense LU back-solve.

9.4 Shape optimization through a differentiable mesh

Shape optimization is the cleanest test of the differentiable mesh: the design variable enters only the geometry, so the drag gradient must thread the entire curved-element solver. We demonstrate it on two deformation families of the Schäfer–Turek cylinder at $\text{Re} = 20$, a Fourier boundary perturbation and a conformal map, that share a single differentiable mesh constructor.

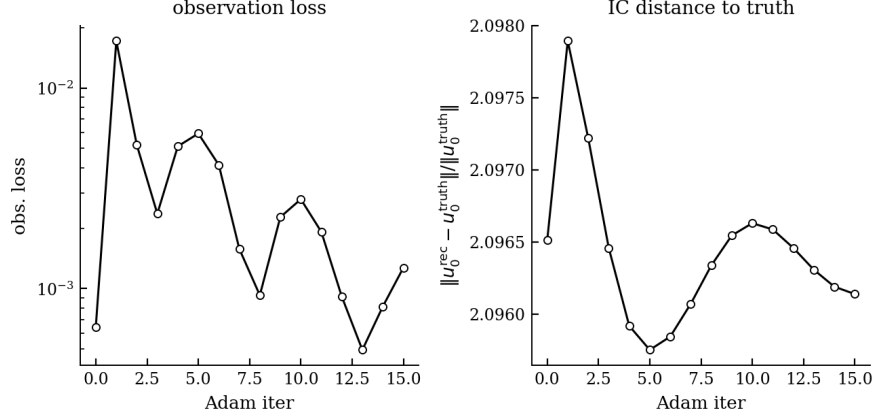


Figure 10: 4D-VAR initial-condition recovery via reverse-mode AD through a 4-step BDF2 trajectory. Left: observation loss under Adam (non-monotone, $\sim 10^{-3}$ envelope). Right: relative recovered-IC distance to the truth, nearly flat at the underdetermined-manifold floor with 48 observation scalars against 3,354 unknowns. The figure shows the reverse-mode AD pass yields usable gradients through the implicit BDF2 inner solve; recovering the IC quantitatively requires a richer observation operator.

9.4.1 Differentiating through the curved mesh

Both families share one ingredient: a body-fitted curvilinear mesh whose construction is a single differentiable map from a handful of shape parameters p to the element geometry. Starting from a fixed mesh topology with a circular hole, a shape map displaces the boundary nodes (a Fourier perturbation $r(\theta; a)$ for the mode-2 family, or the conformal Karman–Trefftz/Joukowski map for the streamlined family), and a Gordon–Hall transfinite blend propagates that displacement into the near-wall elements, curving them to follow the body (§3.2). A radial window $[r_1, r_2]$ tapers the displacement to zero, so elements beyond r_2 stay straight and the channel walls remain fixed (Figure 11). From the curved node positions the constructor evaluates, at every cubature point, the mapping Jacobian J and the metric terms that enter the curl–curl viscous form and the Piola-mapped BDM basis (§6.3, Figure 1).

Every link in this chain (boundary displacement, transfinite blend, Jacobian and metric evaluation, Piola scaling) is written in JAX, so the geometric sensitivity $\partial(\text{mesh})/\partial p$ comes from the same reverse-mode pass that differentiates the solve. The shape gradient dC_D/dp thus flows from the surface-stress integral, through the steady BDF/Newton solve and the curved-element operators, back through the mesh constructor to p , with no hand-coded mesh-sensitivity or adjoint. The constructor is well-posed only while the curving keeps $J > 0$ everywhere; the optimized conformal mesh in Figure 11 retains $\min J > 0$ (no element tangling), and the box constraint on each family is set precisely to stay in that regime.

9.4.2 Mode-2 boundary family

We parameterize the cylinder boundary in polar form as $r(\theta) = R(1 + a \cos 2\theta)$, a single mode-2 amplitude a that elongates ($a > 0$) or flattens ($a < 0$) the body along the flow while preserving the enclosed area to first order. This is the lowest member of the Fourier family $r(\theta) = R(1 + \sum_j a_j \cos(j\theta + \phi_j))$ and suffices to exercise the full reverse-mode path. We minimize the steady-state drag coefficient,

$$\mathcal{L}(a) = C_D(a), \quad |a| \leq a_{\max}, \quad (30)$$

where C_D is the converged steady drag at $\text{Re} = 20$, the stable and well-defined steady-wake regime of the Schäfer–Turek 2D-1 benchmark, obtained by marching the BDM₂ monolithic solver to steady state and evaluating the surface-stress integral of §8.4. The box constraint $a_{\max} = 0.15$ keeps the body-fitted curved mesh valid; beyond $a \approx 0.16$ the Gordon–Hall curving inverts elements next to the wall. The control is optimized by projected Adam, and the gradient dC_D/da flows end-to-end through the surface integral, the steady BDF/Newton solve, the curved-element Piola maps, and the mesh constructor back to a , with no hand-coded adjoint; it agrees with a central finite difference to 0.1%.

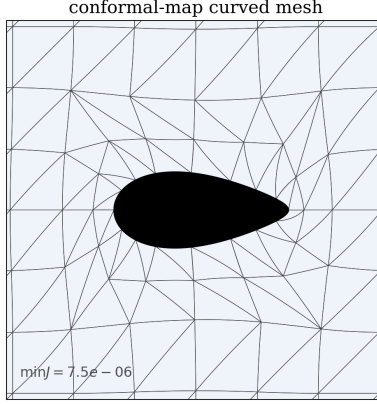


Figure 11: Body-fitted curvilinear mesh at the optimized conformal Kármán–Trefftz/Joukowski shape, rendered at high fidelity (each element mapped from the reference triangle through the isoparametric basis). Near-wall elements curve to follow the body, and the mesh retains a strictly positive mapping Jacobian ($\min J > 0$: no element tangling), the condition the differentiable shape constructor must preserve to remain well-posed. The view is zoomed to the near-body region, so the panel border cuts through the mesh; the mesh extends to the full Schäfer–Turek channel.

Figure 12 (left panel) shows the result. Adam streamlines the cylinder monotonically, reducing the drag from $C_D = 5.22$ for the circle to 4.33, a 17% reduction, before the trace flattens as the boundary saturates the mesh-deformation limit $a = a_{\max}$, where the gradient remains strongly negative. The drag would continue to fall with further streamlining, since this single-mode family has no interior optimum at this Reynolds number, but representing a more slender body would require remeshing rather than curving the existing connectivity.

9.4.3 Conformal-map family

The single mode-2 family of §9.4 is the simplest deformation the differentiable mesh admits; a richer family that is still guaranteed tangle-free is reached through a conformal map. We curve the body-fitted mesh to a circle and then apply a windowed Kármán–Trefftz/Joukowski map, which carries a circle to a smooth, streamlined body and is injective on the circle exterior, so the mapped mesh cannot self-intersect for any parameter value. The map is controlled by a thickness τ and a camber γ (here $\gamma \equiv 0$: a symmetric body, since the goal is a streamlined drag-minimizer, not a lifting section), a trailing-edge rounding κ that holds the mapping Jacobian strictly positive at the otherwise-cusped trailing edge, and a radial window $[r_1, r_2]$ that blends the deformation back to the identity so the channel walls remain fixed. As in §6.3 the entire construction is JAX-traceable, so the sensitivity $dC_D/d(\tau, \gamma)$ flows through the conformal map and the curved-element steady solve with no hand-coded adjoint; it agrees with a central finite difference to under 1%.

We minimize the steady drag at $\text{Re} = 20$ with a soft area penalty that prevents the body from collapsing to a sliver,

$$\mathcal{L}(\tau, \gamma) = C_D(\tau, \gamma) + \lambda_{\text{area}} \left(\frac{A(\tau, \gamma) - A_0}{A_0} \right)^2, \quad (31)$$

with $A_0 = \pi R^2$ the original disc area and $A(\tau, \gamma)$ the enclosed body area in closed form from the conformal map. Figure 12 (center and right panels) shows the result: projected Adam slims the symmetric body monotonically, lowering the drag from $C_D = 4.04$ to 3.45 (a 15% reduction) until the thickness saturates its lower box bound, where the gradient remains strongly negative; as in the mode-2 family, the optimum sits at the edge of the representable shape range. The conformal family reaches a drag well below the mode-2 cylinder, the streamlined body starting beneath the circle’s $C_D = 5.22$ of §9.4, illustrating that a more expressive but still tangle-free shape parameterization requires no change to the differentiable-mesh construction.

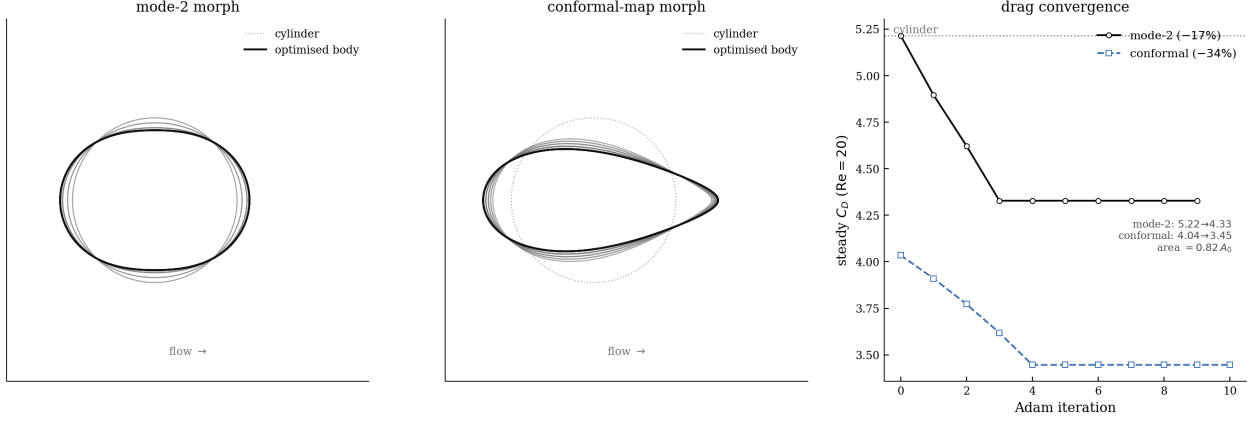


Figure 12: Drag-minimizing shape optimization at $Re = 20$ for two tangle-free deformation families, each reshaping the bluff cylinder into a streamlined low-drag body. *Left:* the single mode-2 family $r(\theta) = R(1 + a \cos 2\theta)$, the body silhouette at each Adam iterate (light initial circle to dark final, $a = 0.15$). *Center:* the conformal Kar  man–Trefftz/Joukowski family parameterized by thickness (camber fixed to zero for a symmetric body) under a soft area penalty: the silhouette morph with the original cylinder dotted; the final body encloses $0.82 A_0$ of the disc area. *Right:* the steady-drag convergence for both families on shared axes against the cylinder reference (dotted), one line per family. The mode-2 body descends $C_D : 5.22 \rightarrow 4.33$ (17% below the cylinder) and the conformal body $4.04 \rightarrow 3.45$ (15% reduction, 34% below the cylinder), each flattening once the shape saturates its parameter bound. The two trajectories begin at different C_D because they start from different geometries: the mode-2 run from the exact circle ($a = 0$), the conformal run from a moderately thick teardrop, since the Joukowski family does not contain the circle. Both gradients are taken end-to-end through the differentiable mesh construction and the curved-element steady solve, each validated against a finite difference (to 0.1% and under 1% respectively).

9.5 Boundary-control wake suppression

A second class of differentiable optimization problem is boundary control: actuate the inflow of the Sch  fer–Turek cylinder channel to suppress the downstream wake of the shed von K  rm  n street at $Re = 100$. The control is a time-varying inlet *profile*

$$v_p(y, t) = v_p^0(y) + \sum_{m=1}^M c_m(t) U_{\max} \sin\left(\frac{2\pi m y}{H}\right), \quad (32)$$

with v_p^0 the baseline parabolic inflow, H the channel height, $U_{\max}$ the inflow peak, and $\{c_m(t)\}$ the design variables, piecewise-constant over N_c knots in time ($M = 4$ modes, $N_c = 24$ knots here). Each perturbation mode $\sin(2\pi m y/H)$ vanishes at the walls and has zero y -mean, so $\int_0^H v_p(y, t) dy = \int_0^H v_p^0(y) dy$ for all t : the volumetric flow rate through the inlet is held constant by construction. This is a hard constraint, not a penalty, and we verify the discrete inlet flux stays fixed to machine precision over the window. The optimizer therefore cannot suppress the wake by throttling the throughput; it can only redistribute momentum across the channel height and pulse it in time. We minimize the time-averaged downstream enstrophy,

$$\mathcal{L}(c) = \left\langle \int_{\Omega_{\text{wake}}} \frac{1}{2} \omega(x, t)^2 dx \right\rangle_{t \in [0, T_w]}, \quad (33)$$

over a fixed downstream box $\Omega_{\text{wake}} \subset \Omega$ behind the cylinder.

Because the shed wake is a self-sustained limit cycle, the control is posed on the developed attractor rather than the transient from rest: a long uncontrolled precursor marches the flow to fully-developed shedding (Strouhal number ≈ 0.2), and its state is saved as the initial condition for the controlled window. The optimization then differentiates only a short window of a few shedding periods ($T_w \approx 1.5$, three periods), which is well-posed because $Re = 100$ shedding is a periodic limit cycle rather than chaos. The restart is

bit-exact: a controlled-window solve continued from the saved state reproduces the uncontrolled trajectory to round-off.

Solver choice. The forward solve uses the block-Schur projection method (§5.2). Its Helmholtz and pressure-Schur factors are assembled once, independently of the control (which enters only the inflow trace), so each step is two LU back-solves; the back-substitution is differentiable, and a scan over the time steps with per-step rematerialization keeps reverse-mode memory bounded independently of the step count. The fully-implicit monolithic solver re-assembles and re-factorizes a dense Newton system each step, so its unrolled differentiable tape grows with the step count and exhausts memory while compiling the gradient of even a sixteen-step BDM₃ trajectory. The projection instead carries an explicit-convection CFL bound (§8.7; here $\Delta t = 5 \times 10^{-4}$), but its lower per-step cost and bounded memory make it the practical choice for differentiable control at this resolution.

Adam on the projection forward (BDM₃, $|\mathcal{T}_h| = 432$, $T_w = 1.5$) reduces the time-averaged downstream enstrophy by 12% ($\mathcal{L} : 269 \rightarrow 236$) over 33 iterations before flattening. The reduction is modest because the actuation is purely upstream and the throughput is fixed, so the controller can only reshape and pulse the oncoming profile rather than weaken the flow. Figure 13 shows the learned profile, the loss history, and the baseline versus controlled wake.

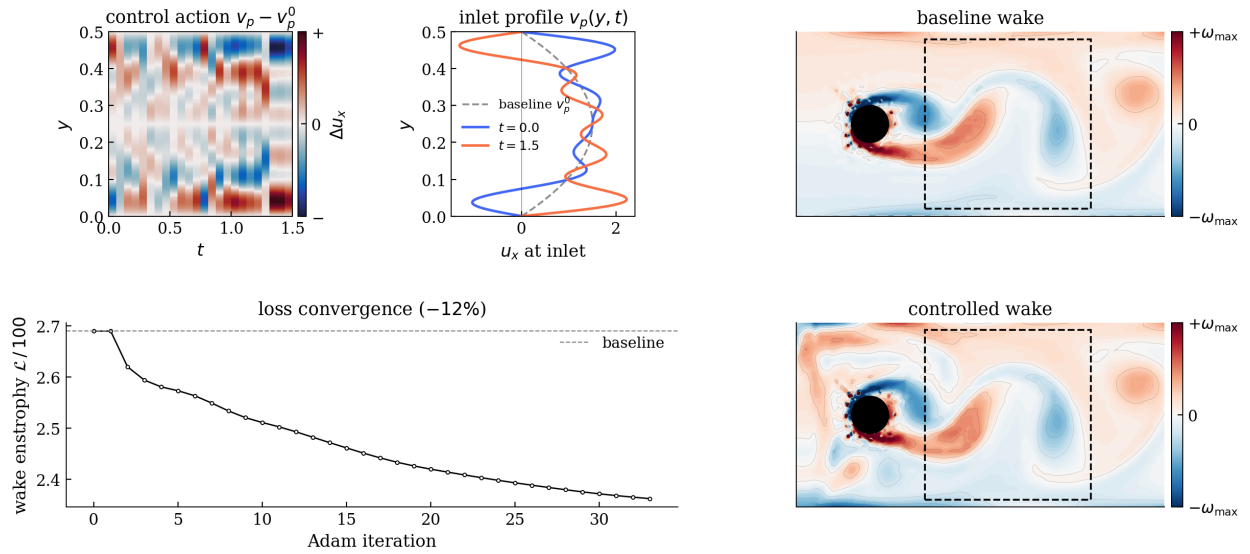


Figure 13: Mass-conserving boundary-control wake suppression on the BDM₃ block-Schur projection solver ($\text{Re} = 100$). Top left: the learned control action $v_p - v_p^0$, the signed inflow perturbation, as a contour over channel height and time on a symmetric scale (red speeds up, blue slows the local inflow), showing the spatiotemporal pulsing the optimizer applies at fixed volumetric flow rate. Top center: the inlet profile $v_p(y, t)$ at $t = 0$ and $t = 1.5$ against the baseline parabola v_p^0 . Bottom left: the time-averaged wake-enstrophy loss, which decreases by 12% under Adam and flattens. Right: the baseline (uncontrolled) and controlled downstream vorticity at a fixed phase of the shedding cycle, on a shared color scale. The forward solve is differentiated end-to-end through the projection’s factored-once LU back-solves, over a short window restarted from a developed-shedding state; the same control is infeasible on the monolithic solver, whose unrolled dense-Newton tape exhausts memory at this order (§9.5).

9.6 Active wake suppression by a body-mounted jet

The upstream inlet control of §9.5 can only reshape the oncoming momentum, which is why its reduction is modest; a far more authoritative actuator sits on the body itself. We place a synthetic jet on the cylinder

surface and impose it as the cylinder Dirichlet trace, $u = v_n(\theta, t) \hat{n}$, with a purely azimuthal normal profile

$$v_n(\theta, t) = U_{\max} \sum_{m=1}^M [a_{c,m}(t) \cos m\theta + a_{s,m}(t) \sin m\theta], \quad (34)$$

where θ is the azimuthal angle around the cylinder and $\{a_{c,m}(t), a_{s,m}(t)\}$ are the design variables, again piecewise-constant in time. Each azimuthal mode integrates to zero around the body, so $\oint v_n ds = 0$ for all t : the jet adds no net mass: it blows on one part of the cylinder and sucks on another, a zero-net-mass-flux (ZNMF) synthetic-jet actuator. As with the inlet control this is a hard constraint of the basis, not a penalty, and we verify the discrete net flux through the cylinder faces stays at machine zero ($\sim 10^{-16}$ relative to $U_{\text{mean}}H$) over the window. To keep the optimum a structured controller rather than a saturated actuator, we add an actuation-effort penalty,

$$\mathcal{L}(a) = \left\langle \int_{\Omega_{\text{wake}}} \frac{1}{2} \omega(x, t)^2 dx \right\rangle_t + \lambda_{\text{eff}} \langle |a(t)|^2 \rangle_t, \quad (35)$$

posed over the same downstream box Ω_{wake} and developed-shedding restart as §9.5, and differentiated through the same factored-once projection solver.

Result. Figure 14 reports a production-resolution run (BDM₃, a window of several shedding periods, ground to flatline over 27 Adam iterations) exercising the full differentiable path: the gradient through the ZNMF jet trace matches a central finite difference to round-off, and Adam descends smoothly to a structured, sub-saturation controller. The body-mounted jet has far more authority than the upstream inlet control: it reduces the time-averaged downstream enstrophy by 72.9%, against the 12% of the mass-conserving inlet profile, and the instantaneous wake collapses progressively as the controller ramps, the downstream box clearing of coherent shedding by the end of the window. The window inherits the finite-horizon caveat of §9.5: part of the late-window suppression is the controller borrowing against a finite horizon.

10 Discussion

The four applications of §9, namely the tuning of the solver’s own numerical parameters, the control of a wake through a time-dependent boundary condition, the optimization of a body’s shape, and the recovery of an unknown initial state from sparse observations, are not four separate methods but a single mechanism applied four times. Any scalar that can be written as a differentiable function of the simulated trajectory becomes an objective, and a single reverse-mode pass through the unmodified high-order solver returns its gradient with respect to every input at a cost independent of the number of design variables. No per-problem adjoint is derived and no surrogate replaces the discretization: the same pressure-robust BDM_k/ P_{k-1} scheme verified against the Stokes, Kovasznay, Ghia, and Schäfer–Turek benchmarks is the scheme through which the gradients flow. The contribution is therefore less a set of results than a capability, that of a mature, high-order, H(div)-conforming incompressible solver that an optimizer can differentiate through directly.

Because the objective is an arbitrary differentiable functional of the state, the same framework reaches a broad range of engineering and experimental problems with no change to the solver. Lift-to-drag and mixing-efficiency design, actuator and inflow control, parameter and geometry inference from experimental (for example PIV or pressure-tap) measurements, and sensitivity and uncertainty studies all reduce to specifying a loss and evaluating its gradient. The hand-written tangent-linear and adjoint code that each new objective has historically demanded is thus paid down once, for all objectives.

A further consequence of the implementation follows from the choice of JAX, a mainstream machine-learning framework with a mature neural-network and optimization ecosystem (Flax [15]/Equinox [20], Optax [9]): a learned component such as a closure, a controller, or a corrector can share a single differentiable program with the PDE discretization, differentiated by one automatic-differentiation system and trained with the same tooling. This makes it natural to couple machine-learning and reinforcement-learning models directly to the solver, whether to learn a turbulence or stabilization closure embedded in the residual, to train a control policy that acts on boundary or body forcing, or to augment the discretization with a learned correction. Autodiff-enabled scientific-computing frameworks built primarily for PDE-constrained optimization and

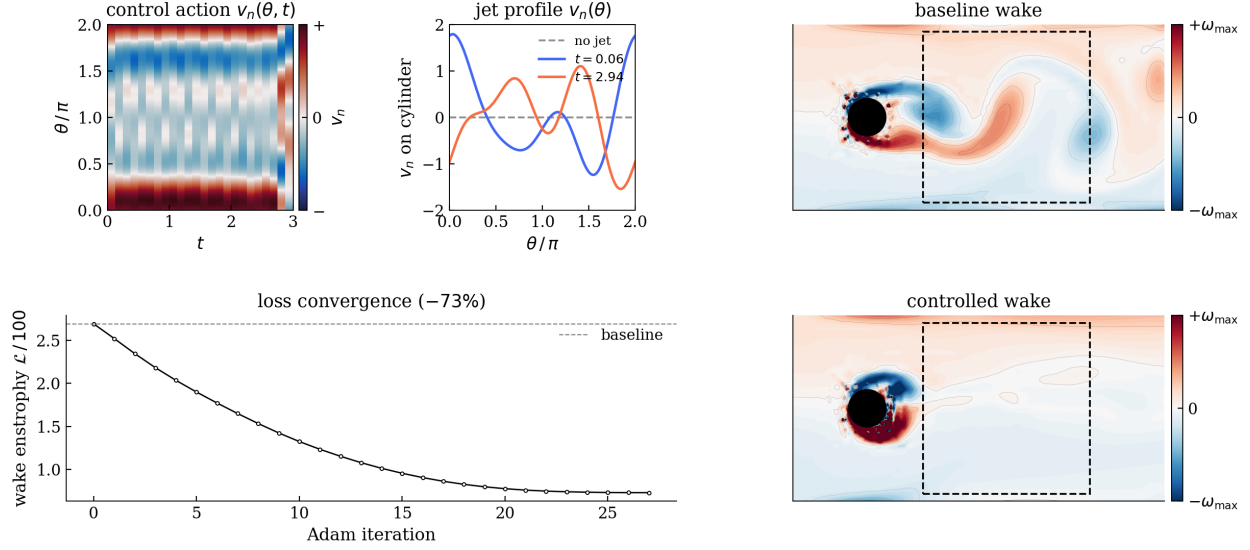


Figure 14: Active wake suppression by a zero-net-mass-flux synthetic jet on the cylinder surface ($\text{Re} = 100$, BDM_3). Top left: the learned control action $v_n(\theta, t)$, the azimuthal jet velocity around the body over time on a symmetric scale (red blows out, blue sucks in), integrating to zero around the cylinder at every instant. Top center: the azimuthal jet profile $v_n(\theta)$ early and late in the window. Bottom left: the time-averaged wake-enstrophy loss, descending 72.9% under Adam and flattening. Right: the baseline (uncontrolled) versus controlled downstream vorticity near the end of the window, on a shared color scale, with the downstream enstrophy-integral region (the control objective) drawn dashed; the jet concentrates vorticity in a thin layer at the body and clears the downstream box. The gradient flows end-to-end through the projection solver’s factored LU back-solves and the ZNMF jet trace, over a window restarted from a developed-shedding state.

sensitivity analysis instead obtain gradients by assembling the discrete adjoint or tangent-linear equations, and coupling them to neural networks bridges two distinct automatic-differentiation systems; even the dedicated finite-element and deep-learning interfaces now available [4] compose a finite-element adjoint with a separate machine-learning autograd rather than expressing both within a single trace. Sharing one trace with the deep-learning stack is what makes such learned closures and learning-based solver control natural to express in this setting.

The results reported here are two-dimensional. The BDM_k/P_{k-1} scheme and its differentiable formulation extend to tetrahedral three-dimensional meshes without formal obstruction; the engineering involved, comprising tetrahedral Piola maps, triangular face degrees of freedom, and curved non-planar boundaries, is substantial but not conceptual. The differentiable applications were run on short integration windows and at modest resolution, and a production deployment would lengthen the windows, raise the BDM order and mesh resolution, and address the gradient-checkpointing memory that bounds trajectory length on commodity hardware; these are matters of scale rather than of principle. The projection-versus-monolithic trade-off of §8.7 is likewise an explicit-convection CFL constraint rather than a fundamental limitation, since an implicit treatment of convection inside the projection velocity solve closes the stability gap and the framework is structured to accommodate it.

The most immediate opportunity is to let the differentiability drive the parts of a solver that are still set by hand. The parameter tuning of §9.1 optimizes differentiable solver objectives directly, and coupling the same exposed solver to a reinforcement-learning agent or other external reward signal would extend this to quantities that are expensive or non-differentiable to evaluate, and to online adaptation in which the time step, penalty coefficients, smoother weights, and preconditioner parameters are adjusted during a run in response to the evolving flow rather than fixed in advance. A differentiable, machine-learning-native solver is what such a controller requires, turning the heuristic, hand-tuned layer of a modern high-order method into a learnable one. Integrating reinforcement-learning approaches to augment the solver’s numerical capabilities, together with the extension to three-dimensional problems, is the subject of ongoing research.

References

- [1] Grégoire Allaire, Charles Dapogny, and Pascal Frey. Shape optimization with a level set based mesh evolution method. *Computer Methods in Applied Mechanics and Engineering*, 282:22–53, 2014.
- [2] Wolfgang Bangerth, Ralf Hartmann, and Guido Kanschat. deal.II — a general-purpose object-oriented finite element library. *ACM Transactions on Mathematical Software*, 33(4):24:1–24:27, 2007.
- [3] Deniz A. Bezgin, Aaron B. Buhendwa, and Nikolaus A. Adams. JAX-Fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows. *Computer Physics Communications*, 282:108527, 2023.
- [4] Nacime Bouziani and David A. Ham. Physics-driven machine learning models coupling PyTorch and Firedrake. In *ICLR 2023 Workshop on Physics for Machine Learning*, 2023. arXiv:2303.06871.
- [5] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [6] F. Brezzi, J. Douglas, and L. D. Marini. Two families of mixed finite elements for second order elliptic problems. *Numerische Mathematik*, 47(2):217–235, 1985.
- [7] F. Brezzi and J. Pitkäranta. On the stabilization of finite element approximations of the Stokes equations. *Efficient Solutions of Elliptic Systems, Notes Numer. Fluid Mech.*, 10:11–19, 1984.
- [8] J. Cahouet and J.-P. Chabard. Some fast 3D finite element solvers for the generalized Stokes problem. *International Journal for Numerical Methods in Fluids*, 8:869–895, 1988.
- [9] DeepMind, Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Tom Hennigan, Matteo Hessel, Steven Kapturowski, Thomas Keck, Iurii Kemaev, et al. The DeepMind JAX Ecosystem, 2020.
- [10] Niklas Fehn, Martin Kronbichler, and Gert Lube. From anomalous dissipation through Euler singularities to stabilized finite element methods for turbulent flows. *Flow, Turbulence and Combustion*, 115:347–388, 2025.
- [11] Niklas Fehn, Peter Munch, Wolfgang A. Wall, and Martin Kronbichler. ExaDG: High-order discontinuous Galerkin for the exa-scale. In *Software for Exascale Computing – SPPEXA 2016–2019*, volume 136 of *Lecture Notes in Computational Science and Engineering*, pages 189–224. Springer, 2020.
- [12] Niklas Fehn, Wolfgang A. Wall, and Martin Kronbichler. On the stability of projection methods for the incompressible Navier–Stokes equations based on high-order discontinuous Galerkin discretizations. *Journal of Computational Physics*, 351:392–421, 2017. Identifies small- Δt instability of DG dual splitting; introduces the consistent-splitting alternative.
- [13] Niklas Fehn, Wolfgang A. Wall, and Martin Kronbichler. Robust and efficient discontinuous Galerkin methods for under-resolved turbulent incompressible flows. *Journal of Computational Physics*, 372:667–693, 2018. Introduces the divergence and continuity penalty post-projection used in this work.
- [14] Daniel Greenfeld, Meirav Galun, Ronen Basri, Irad Yavneh, and Ron Kimmel. Learning to optimize multigrid PDE solvers. In *International Conference on Machine Learning (ICML)*, pages 2415–2423, 2019.
- [15] Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for JAX, 2024.
- [16] Jan S. Hesthaven and Tim Warburton. *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*, volume 54 of *Texts in Applied Mathematics*. Springer, 2008.
- [17] Volker John, Alexander Linke, Christian Merdon, Michael Neilan, and Leo G. Rebholz. On the divergence constraint in mixed finite element methods for incompressible flows. *SIAM Review*, 59(3):492–544, 2017.
- [18] Shinhoo Kang and Emil M. Constantinescu. Differentiable DG with neural operator source term correction. *arXiv preprint arXiv:2310.18897*, 2023.
- [19] George Em Karniadakis, Moshe Israeli, and Steven A. Orszag. High-order splitting methods for the incompressible Navier–Stokes equations. *Journal of Computational Physics*, 97(2):414–443, 1991.
- [20] Patrick Kidger and Cristian Garcia. Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming workshop at Neural Information Processing Systems*, 2021.
- [21] Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning-accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021.
- [22] François-Xavier Le Dimet and Olivier Talagrand. Variational algorithms for analysis and assimilation of meteorological observations: theoretical aspects. *Tellus A*, 38(2):97–110, 1986.

- [23] Alexander Linke. On the role of the Helmholtz decomposition in mixed methods for incompressible flows and a new variational crime. *Computer Methods in Applied Mechanics and Engineering*, 268:782–800, 2014.
- [24] Siddhartha Mishra and Roberto Molinaro. Estimates on the generalization error of physics-informed neural networks for approximating pdes. *IMA Journal of Numerical Analysis*, 43:1–43, 2023.
- [25] Sebastian K. Mitusch, Simon W. Funke, and Jørgen S. Dokken. dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake. *Journal of Open Source Software*, 4(38):1292, 2019.
- [26] A. H. Stroud. *Approximate Calculation of Multiple Integrals*. Prentice-Hall, Englewood Cliffs, NJ, 1971.
- [27] Olivier Talagrand and Philippe Courtier. Variational assimilation of meteorological observations with the adjoint vorticity equation. *Q. J. Roy. Meteor. Soc.*, 113:1311–1328, 1987.